Institut Supérieur CPTEC • Support de cours

MATHÉMATIQUES APPLIQUÉES À L'INFORMATIQUE

Des ensembles et de la logique aux images numériques

Support de cours de l'étudiant : six chapitres avec exemples résolus

Licence 1 & Licence 2

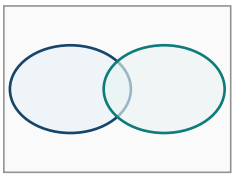
Année académique 2026–2027

Déc.	Binaire	Hexa
9	1001	9
11	1011	B
26	11010	1A

Numération

p	q	$p \Rightarrow q$
V	V	V
V	F	**F**
F	V	V
F	F	V

Logique



$A \cup B$, $A \cap B$ et $\overline{A}$

Ensembles

Institut Supérieur CPTEC

Agréé par arrêté N° 00.50/2024 • Avedji, Lomé (Togo) • +228 70 78 78 18 • 96 51 05 50 • cptec228togo@gmail.com

Ounimborbitibou DJABON

Titulaire de deux Masters : Sciences Mathématiques (AIMS Ghana)
et Physique Théorique (Université de Lomé)

Chercheur en Information et Communication Quantique

+228 92 39 97 21 • djabon@aims.edu.gh • github.com/djabelo712

Institut Supérieur CPTEC • Support de cours

MATHÉMATIQUES APPLIQUÉES À L'INFORMATIQUE

Support de cours de l'étudiant

Ensembles • Logique et algèbre de Boole • Systèmes de numération
Données en mémoire • Mesures des angles • Vecteurs et images numériques

L'institut

CPTEC Business School
Institut Supérieur

Agréé par l'État N° 00.50/2024

Avedji, Lomé (Togo)
Rue en face de l'Église Évangélique Presbytérienne du Togo (EEPT), au niveau de SUNU Assurance

+228 70 78 78 18 • 96 51 05 50
cptec228togo@gmail.com

L'enseignant

Ounimborbitibou DJABON

Titulaire de deux Masters :
Sciences Mathématiques (AIMS Ghana)
Physique Théorique (Université de Lomé)

Chercheur en Information et Communication Quantique

+228 92 39 97 21
djabon@aims.edu.gh
github.com/djabelo712

Niveau : Licence 1 & Licence 2

Année académique 2026–2027

Objectifs du module

Objectif général : donner aux étudiants les méthodes mathématiques directement utilisées en informatique, des ensembles et de la logique jusqu'à la représentation des données et des images dans la machine.

Objectifs spécifiques : à l'issue de cet enseignement, l'étudiant doit être capable de :

- définir les ensembles, les représenter et appliquer les opérations entre ensembles ;
- raisonner avec la logique des propositions et simplifier avec l'algèbre de Boole ;
- convertir les nombres entre les bases 2, 8, 10 et 16 et y effectuer les opérations ;
- représenter les nombres et les données dans la mémoire d'un ordinateur ;
- mesurer les angles en degrés et en radians et résoudre les triangles ;
- manipuler les vecteurs et les matrices ;
- calculer la taille, le codage des couleurs et le poids d'une image numérique.

Table des matières

1 Les ensembles 1
- 1 Notions d'ensembles 1
- 2 Diagramme de Venn 2
- 3 Lien entre deux ensembles 4
- 4 Propriétés des opérations sur les ensembles 6
- 5 Simplification à l'aide des propriétés 7

2 Logique et algèbre de Boole 9
- 1 Opérations logiques 9
- 2 Énoncés conditionnels et biconditionnels 11
- 3 Les opérations bit à bit 12
- 4 Circuits et portes logiques 14
- 5 Propriétés des opérations logiques 16
- 6 Simplification booléenne 17

3 Systèmes de numération et opérations 21
- 1 Systèmes de numération 21
- 2 Conversions 23
- 3 Opérations dans les systèmes 26
- 4 Codes binaires 27

4 Données en mémoire 29
- 1 Représentation des nombres en mémoire 29
- 2 Mémoire d'un ordinateur 30
- 3 Limites de précision 32
- 4 Choix de la bonne représentation 33

5 Mesures des angles 35
- 1 Mesure des angles 35
- 2 Les triangles 36
- 3 Les vecteurs 38

6 Les vecteurs 41
- 1 Définition de vecteurs et de matrices 41
- 2 Image vectorielle et image matricielle 42
- 3 Taille et dimension d'une image numérique 43
- 4 Codage des couleurs 45
- 5 Poids d'une image numérique 46

Chapitre 1

Les ensembles

En quelques lignes

L'ensemble est la brique élémentaire de toutes les mathématiques appliquées à l'informatique : les tables d'une base de données, les fichiers d'un répertoire, les étudiants d'une promotion ou les adresses d'un réseau local se décrivent tous comme des collections d'objets bien définis. Ce chapitre fixe le vocabulaire (appartenance, représentation, cardinal), puis construit les trois opérations fondamentales (union, intersection, complémentaire) et leurs propriétés, avec le diagramme de Venn comme support visuel. Il prépare directement le chapitre 2, où les mêmes idées deviennent des propositions logiques, et le chapitre 3, où compter les codages possibles revient à compter les sous-ensembles d'un ensemble fini.

1 Notions d'ensembles

Définition 1.1 : ensemble, appartenance

Un *ensemble* est une collection d'objets, appelés *éléments*, bien définie : pour tout objet x, on peut dire sans ambiguïté si x appartient à l'ensemble (on écrit $x \in A$) ou n'y appartient pas (on écrit $x \notin A$). Un ensemble peut se noter en extension, en écrivant tous ses éléments entre accolades, ou en compréhension, en donnant la propriété que ses éléments vérifient.

Les deux écritures suivantes désignent le même ensemble des entiers de 3 à 9 :

$$A = \{3, 4, 5, 6, 7, 8, 9\} \qquad \text{et} \qquad A = \{x \in \mathbb{N} \mid 3 \leqslant x \leqslant 9\}.$$

L'écriture en extension est directe mais ne convient qu'aux ensembles finis et assez courts ; l'écriture en compréhension est la seule possible pour un ensemble infini comme $\mathbb{N}$ lui-même. L'ordre des éléments dans les accolades n'a aucune importance, et écrire deux fois le même élément ne change rien : $\{1, 2, 2, 3\}$ est exactement l'ensemble $\{1, 2, 3\}$.

Définition 1.2 : cardinal, ensemble fini

Un ensemble est *fini* lorsque ses éléments peuvent être comptés. Le nombre de ses éléments s'appelle son *cardinal* et se note $|A|$. Le *vide*, noté $\varnothing$, est l'ensemble qui ne contient aucun élément : $|\varnothing| = 0$. Un ensemble à un seul élément s'appelle un *singleton*.

Exemple résolu 1.1 : représenter et compter

Soit $A = \{x \in \mathbb{N} \mid 3 \leqslant x \leqslant 9\}$ et $B = \{c, t, r, p\}$, l'ensemble des lettres du mot « cptr ». L'ensemble A s'écrit en extension $\{3, 4, 5, 6, 7, 8, 9\}$ et son cardinal vaut $|A| = 7$; l'ensemble B contient quatre lettres distinctes, donc $|B| = 4$. Enfin $5 \in A$, $10 \notin A$ et la lettre $t \in B$, alors que $z \notin B$.

Les ensembles de nombres que nous utiliserons pendant tout le semestre forment une chaîne d'emboîtement : chaque ligne du tableau 1.1 contient la précédente.

Notation	Ensemble	Exemples d'éléments
$\mathbb{N}$	entiers naturels : $0, 1, 2, 3, \ldots$	0 ; 17 ; 256
$\mathbb{Z}$	entiers relatifs : $\ldots, -2, -1, 0, 1, 2, \ldots$	-7 ; 0 ; 1024
$\mathbb{D}$	nombres décimaux, écriture finie après la virgule	-7 ; $0{,}75$; $2{,}5$
$\mathbb{Q}$	nombres rationnels, quotients d'entiers	-7 ; $\frac{2}{3}$; $0{,}75$
$\mathbb{R}$	nombres réels, rationnels et irrationnels	$\sqrt{2}$; π ; $0{,}75$

Table 1.1 – Les ensembles de nombres usuels, emboîtés de $\mathbb{N}$ vers $\mathbb{R}$

Le tableau 1.1 se lit de gauche à droite comme des poupées gigognes : tout entier naturel est un entier relatif, tout entier relatif est un décimal, tout décimal est un rationnel, tout rationnel est un réel. Les décimaux $\mathbb{D}$ sont les nombres dont l'écriture s'arrête après la virgule, comme $0{,}75$ ou $2{,}5$: chacun s'écrit comme un quotient d'entiers par une puissance de 10, et la fraction $\frac{1}{3}$, dont les décimales se répètent sans fin, n'en fait pas partie. La figure 1.1 illustre cet emboîtement. Une petite difficulté de vocabulaire mérite d'être signalée dès maintenant : le nombre $\frac{4}{2}$, bien qu'écrit comme un quotient, vaut exactement 2 et appartient donc à $\mathbb{N}$; c'est la valeur qui décide de l'appartenance, non l'écriture. Les nombres $\sqrt{2}$, π ou $0{,}123456789101112\ldots$ sont, eux, irrationnels : leurs décimales ne se répètent jamais de façon périodique, et aucune paire d'entiers ne les représente exactement.

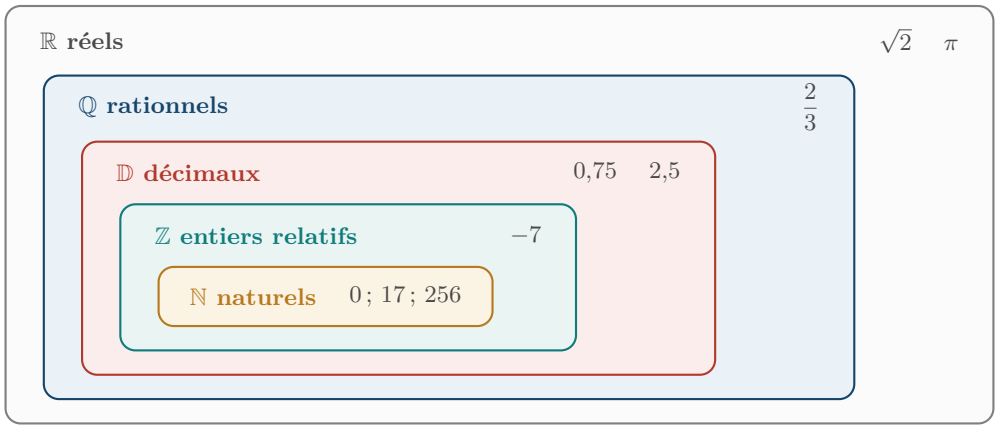


Figure 1.1 – Les ensembles de nombres s'emboîtent : $\mathbb{N} \subset \mathbb{Z} \subset \mathbb{D} \subset \mathbb{Q} \subset \mathbb{R}$

Exemple résolu 1.2 : classer des nombres

Rangeons les nombres -7, $\frac{2}{3}$, $0{,}75$, 0, $\sqrt{2}$, 15 dans les bons ensembles. Les nombres -7, 0 et 15 sont des entiers relatifs, donc ils appartiennent à $\mathbb{Z}$ et aussi à $\mathbb{N}$ pour 0 et 15 ; le nombre $0{,}75$ est décimal, car $0{,}75 = \frac{75}{100}$; le nombre $\frac{2}{3}$ est rationnel sans être décimal, ses décimales se répétant sans fin ; le nombre $\sqrt{2}$ est réel mais irrationnel, car aucun quotient d'entiers ne vaut exactement $\sqrt{2}$. On a donc $0, 15 \in \mathbb{N}$; $-7 \in \mathbb{Z}$; $0{,}75 \in \mathbb{D}$; $\frac{2}{3} \in \mathbb{Q}$; $\sqrt{2} \in \mathbb{R}$ mais $\sqrt{2} \notin \mathbb{Q}$.

2 Diagramme de Venn

Le diagramme de Venn représente un ensemble par une courbe fermée dessinée à l'intérieur d'un rectangle : le rectangle figure l'*univers* U, c'est-à-dire l'ensemble de tous les objets considérés

pendant l'exercice, et chaque courbe regroupe les éléments d'un ensemble. Cette représentation visuelle permet de lire les trois opérations fondamentales avant même de les calculer.

Définition 1.3 : union, intersection, complémentaire

Soient A et B deux ensembles inclus dans un univers U.

- l'*union* $A \cup B$ est l'ensemble des éléments qui appartiennent à A *ou* à B (au moins l'un des deux) ;
- l'*intersection* $A \cap B$ est l'ensemble des éléments qui appartiennent à A *et* à B (aux deux à la fois) ;
- le *complémentaire* de A, noté $\overline{A}$, est l'ensemble des éléments de U qui n'appartiennent pas à A.

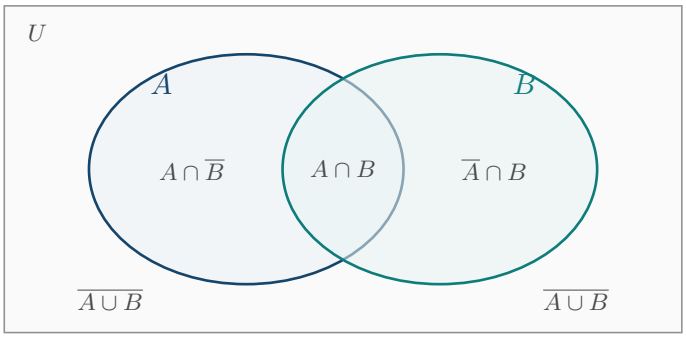


Figure 1.2 – Diagramme de Venn de deux ensembles sécants : chaque région porte un nom, et $\overline{A} \cap \overline{B} = \overline{A \cup B}$

La figure 1.2 se lit ainsi : l'union $A \cup B$ couvre les deux courbes réunies, l'intersection $A \cap B$ est le lentille central où elles se chevauchent, et le complémentaire de l'union est la zone du rectangle extérieure aux deux courbes. Pour compter les éléments d'un diagramme, on commence toujours par remplir la zone centrale $A \cap B$, puis on en déduit les régions A seul et B seul par soustraction.

Exemple résolu 1.3 : répartir 120 étudiants de CPTEC

Dans une promotion de 120 étudiants de l'Institut Supérieur CPTEC, 65 suivent le module de programmation, 50 suivent le module de réseaux et 25 suivent les deux. Remplissons le diagramme de Venn, l'univers étant la promotion entière. La zone centrale vaut $|P \cap R| = 25$. La zone « programmation seulement » vaut $65 - 25 = 40$, la zone « réseaux seulement » vaut $50 - 25 = 25$, et l'union couvre donc $40 + 25 + 25 = 90$ étudiants. Il reste $120 - 90 = 30$ étudiants qui ne suivent aucun des deux modules : ils occupent la zone $\overline{P \cup R}$. On vérifie au passage la formule $|P \cup R| = |P| + |R| - |P \cap R| = 65 + 50 - 25 = 90$.

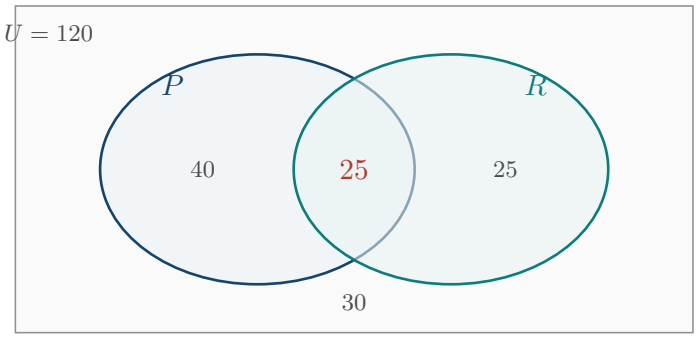


Figure 1.3 – Répartition des 120 étudiants : 90 suivent au moins un des deux modules

Exemple résolu 1.4 : compter avec le complémentaire

Dans une classe de 40 étudiants, 18 participent au club de mathématiques (M) et 23 au club d'informatique (I), 12 étudiants participant aux deux clubs. L'union couvre $|M \cup I| = 18 + 23 - 12 = 29$ étudiants ; le complémentaire $\overline{M \cup I}$ compte donc $40 - 29 = 11$ étudiants qui ne participent à aucun club. Les lois de De Morgan le confirment : ne participer à aucun club, c'est ne pas être au club de mathématiques *et* ne pas être au club d'informatique, soit $\overline{M} \cap \overline{I} = \overline{M \cup I}$, et les deux lectures donnent bien les mêmes 11 étudiants.

Les trois opérations se composent : on peut former $\overline{A \cap B}$, $(A \cup B) \cap C$ et bien d'autres. Le diagramme de Venn suit sans effort : avec trois ensembles, le rectangle se partage en huit régions, comme le montre la figure 1.4. Chaque région correspond à une combinaison d'appartenances (dans A mais ni dans B ni dans C, dans A et B mais pas dans C, et ainsi de suite), et le comptage se fait toujours du centre vers l'extérieur : d'abord le noyau $A \cap B \cap C$, puis les intersections doubles, puis les zones simples.

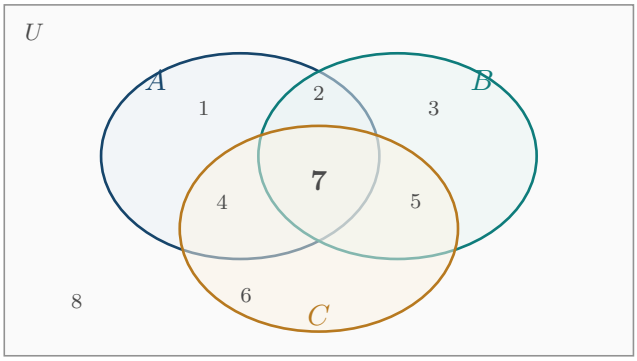


Figure 1.4 – Trois ensembles sécants : huit régions numérotées, dont le noyau $A \cap B \cap C$ (région 7)

3 Lien entre deux ensembles

Deux ensembles peuvent être comparés de trois manières : l'un peut être contenu dans l'autre, ils peuvent être égaux, ou ils peuvent ne rien partager.

Définition 1.4 : inclusion, égalité, ensembles disjoints

Soient A et B deux ensembles.

- A est *inclus* dans B, on écrit $A \subset B$, lorsque tout élément de A appartient aussi à B ; on dit que A est un *sous-ensemble* de B.

- A et B sont *égaux*, on écrit $A = B$, lorsqu'ils ont exactement les mêmes éléments, ce qui revient à avoir à la fois $A \subset B$ et $B \subset A$.
- A et B sont *disjoints* lorsque $A \cap B = \varnothing$: aucun élément commun.

Propriété 1.1 : caractérisation de l'inclusion

Pour tous ensembles A et B inclus dans un univers U :

$$A \subset B \iff A \cap B = A \iff A \cup B = B.$$

Ces deux égalités sont très utilisées : elles traduisent l'inclusion en une relation entre les opérations des sections précédentes.

Par exemple, avec $A = \{1, 2, 3\}$, $B = \{1, 2, 3, 4, 5\}$ et $C = \{6, 7\}$, on a $A \subset B$ car les trois éléments de A figurent dans B, et $A \cap C = \varnothing$: les ensembles A et C sont disjoints. En revanche $B \not\subset A$ car $4 \in B$ alors que $4 \notin A$.

Une confusion classique sépare l'*appartenance* de l'*inclusion* : l'élément 2 appartient à $\{1, 2, 3\}$, on écrit $2 \in \{1, 2, 3\}$, tandis que l'ensemble $\{2\}$ est inclus dans $\{1, 2, 3\}$, on écrit $\{2\} \subset \{1, 2, 3\}$. L'appartenance relie un objet à un ensemble, l'inclusion relie un ensemble à un ensemble ; écrire $2 \subset \{1, 2, 3\}$ ou $\{2\} \in \{1, 2, 3\}$ n'a pas de sens ici, et cette distinction de niveaux évite bien des erreurs de rédaction.

Définition 1.5 : produit cartésien

Le *produit cartésien* de deux ensembles A et B, noté $A \times B$, est l'ensemble des couples (x, y) avec $x \in A$ et $y \in B$. Lorsque A et B sont finis, son cardinal vaut $|A \times B| = |A| \times |B|$.

Exemple résolu 1.5 : tester l'inclusion et la disjonction

Avec $A = \{1, 2, 3\}$, $B = \{1, 2, 3, 4, 5\}$ et $C = \{6, 7\}$, on a $A \subset B$ car les trois éléments de A figurent dans B, et l'on vérifie la caractérisation : $A \cap B = \{1, 2, 3\} = A$ et $A \cup B = \{1, 2, 3, 4, 5\} = B$. Les ensembles A et C sont disjoints, puisque $A \cap C = \varnothing$. En revanche $B \not\subset A$ car $4 \in B$ alors que $4 \notin A$: un seul contre-exemple suffit à ruiner une inclusion.

Un couple (x, y) est une paire ordonnée : $(1, 2)$ et $(2, 1)$ sont deux couples distincts. Cette notion décrit toutes les combinaisons possibles de deux choix successifs : un identifiant et un mot de passe, une ligne et une colonne d'une feuille de calcul, un utilisateur et une machine d'un réseau. Ainsi, un service qui propose 3 identifiants possibles et 2 mots de passe autorisés offre $3 \times 2 = 6$ couples identifiant-mot de passe : $|A \times B| = 3 \times 2 = 6$.

Exemple résolu 1.6 : compter les sous-ensembles

Pour construire un sous-ensemble, chaque élément de A offre exactement deux choix : être pris ou être laissé. Un sous-ensemble de A se décrit donc entièrement par un mot de trois bits, qui marque par 1 les éléments pris et par 0 les éléments laissés :

a pris ?	b pris ?	c pris ?	Sous-ensemble	Mot binaire
0	0	0	$\varnothing$	000
1	0	0	$\{a\}$	100
1	1	0	$\{a, b\}$	110
1	1	1	$\{a, b, c\}$	111

Table 1.2 – Chaque sous-ensemble correspond à un mot binaire

Il y a donc $2 \times 2 \times 2 = 2^3 = 8$ sous-ensembles, dont quatre sont listés dans le tableau 1.2 et les quatre autres sont $\{b\}$, $\{c\}$, $\{a, c\}$ et $\{b, c\}$. Le résultat est général : un ensemble à n éléments possède 2^n sous-ensembles. Ce nombre est exactement celui des codages binaires sur n bits, étudié au chapitre 3 : compter les codages, c'est compter les sous-ensembles.

4 Propriétés des opérations sur les ensembles

Les opérations $\cup$, $\cap$ et le complémentaire obéissent à des lois fixes, rassemblées dans le tableau 1.3. Ces lois se lisent en remplaçant mentalement A et B par des zones du diagramme de Venn ; chacune se traduit par un dessin où les deux membres de l'égalité recouvrent exactement la même région.

Loi	Première forme	Seconde forme
Commutativité	$A \cup B = B \cup A$	$A \cap B = B \cap A$
Associativité	$(A \cup B) \cup C = A \cup (B \cup C)$	$(A \cap B) \cap C = A \cap (B \cap C)$
Distributivité	$A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$	$A \cup (B \cap C) = (A \cup B) \cap (A \cup C)$
Idempotence	$A \cup A = A$	$A \cap A = A$
Absorption	$A \cup (A \cap B) = A$	$A \cap (A \cup B) = A$
Involutivité	$\overline{\overline{A}} = A$	
Lois de De Morgan	$\overline{A \cup B} = \overline{A} \cap \overline{B}$	$\overline{A \cap B} = \overline{A} \cup \overline{B}$
Neutre et absorbant	$A \cup \varnothing = A,\ \ A \cup U = U$	$A \cap U = A,\ \ A \cap \varnothing = \varnothing$

Table 1.3 – Lois algébriques des opérations sur les ensembles

Chaque loi se lit sur le diagramme de Venn en coloriant les deux membres : ils recouvrent exactement la même région. L'absorption $A \cup (A \cap B) = A$ se lit en un coup d'œil : la zone de $A \cap B$ est déjà à l'intérieur de A, donc l'union ne l'agrandit pas. L'idempotence $A \cup A = A$ dit simplement qu'unionner un ensemble avec lui-même ne change rien, de même que la distributivité gauche ou droite se vérifie en superposant deux coloriages. Cette lecture graphique ne remplace pas le calcul : elle l'oriente, en suggérant quelle loi appliquer avant de le mener.

Les lois de De Morgan méritent une lecture attentive : le complémentaire d'une union est l'intersection des complémentaires, et le complémentaire d'une intersection est l'union des complémentaires. En français courant : « ne suivre ni programmation ni réseaux » revient à « ne pas suivre programmation *et* ne pas suivre réseaux ». On les retrouve mot pour mot au chapitre 2 sous la forme des lois de De Morgan de la logique.

Exemple résolu 1.7 : vérifier une distributivité sur des ensembles concrets

Prenons $A = \{1, 2, 3, 4\}$, $B = \{3, 4, 5\}$ et $C = \{4, 5, 6\}$ et vérifions $A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$. D'un côté, $B \cup C = \{3, 4, 5, 6\}$, donc $A \cap (B \cup C) = \{3, 4\}$. De l'autre, $A \cap B = \{3, 4\}$ et $A \cap C = \{4\}$, donc $(A \cap B) \cup (A \cap C) = \{3, 4\} \cup \{4\} = \{3, 4\}$. Les deux membres coïncident : la loi est vérifiée sur cet exemple, comme elle l'est toujours.

Remarque

[la même algèbre pour les ensembles et pour la logique] Le tableau 1.3 et le tableau des connecteurs du chapitre 2 se ressemblent fort : union et intersection jouent les rôles du « ou » et du « et », le complémentaire joue le rôle de la négation, et les lois portent les mêmes noms. Ce n'est pas un hasard : « x appartient à $A \cap B$ » est une phrase qui dit « $x \in A$ *et* $x \in B$ ». Le chapitre 2 exploitera ce parallèle jusqu'au bout, et l'algèbre de Boole le formalisera pour les circuits.

Exemple résolu 1.8 : vérifier une loi de De Morgan sur des ensembles concrets

Dans l'univers $U = \{1, 2, 3, 4, 5, 6, 7, 8\}$, prenons $A = \{2, 3, 5\}$ et $B = \{1, 2, 4, 5, 6\}$ et vérifions $\overline{A \cup B} = \overline{A} \cap \overline{B}$. D'un côté, $A \cup B = \{1, 2, 3, 4, 5, 6\}$, donc $\overline{A \cup B} = \{7, 8\}$. De l'autre, $\overline{A} = \{1, 4, 6, 7, 8\}$ et $\overline{B} = \{3, 7, 8\}$, donc $\overline{A} \cap \overline{B} = \{7, 8\}$. Les deux membres coïncident : la loi est vérifiée sur cet exemple, et elle l'est pour tous les autres.

Remarque

Une loi du tableau 1.3 se retient d'autant mieux qu'on l'a vue échouer une fois dans son usage. Par exemple, la distributivité de $\cup$ sur $\cap$ est souvent oubliée par les étudiants : $A \cup (B \cap C)$ n'est *pas* égal en général à $(A \cup B) \cap C$, mais à $(A \cup B) \cap (A \cup C)$. En cas de doute, refaire un petit test numérique comme celui de l'exemple précédent : trois minutes de calcul évitent une erreur d'orientation.

5 Simplification à l'aide des propriétés

Beaucoup d'expressions d'ensembles s'écrivent sous une forme bien plus courte. La démarche est algébrique : on part de l'expression donnée, on repère dans le tableau 1.3 une loi applicable, on la remplace, et l'on recommence jusqu'à ce qu'aucune loi ne s'applique davantage.

Méthode : simplifier une expression d'ensembles

1. éliminer d'abord les complémentaires d'expressions à l'aide des lois de De Morgan ;
2. distribuer $\cap$ sur $\cup$ (ou $\cup$ sur $\cap$) pour faire apparaître les paires $A \cap \overline{A} = \varnothing$ et $A \cup \overline{A} = U$;
3. simplifier à l'aide de l'idempotence, de l'absorption et des éléments neutre et absorbant ;
4. contrôler le résultat en comparant les zones du diagramme de Venn ou en testant sur un petit exemple numérique.

Exemple résolu 1.9 : une simplification pas à pas

Simplifions $E = (A \cap B) \cup (A \cap \overline{B}) \cup (\overline{A} \cap B)$.

$$\begin{aligned}
E &= \big(A \cap (B \cup \overline{B})\big) \cup (\overline{A} \cap B) && \text{distributivité de } \cap \text{ sur } \cup \\
&= (A \cap U) \cup (\overline{A} \cap B) && \text{car } B \cup \overline{B} = U \\
&= A \cup (\overline{A} \cap B) && \text{neutre de } \cap \\
&= (A \cup \overline{A}) \cap (A \cup B) && \text{distributivité de } \cup \text{ sur } \cap \\
&= U \cap (A \cup B) && \text{car } A \cup \overline{A} = U \\
&= A \cup B. && \text{neutre de } \cup
\end{aligned}$$

L'expression de départ, qui mêle six opérations, vaut donc simplement $A \cup B$: un étudiant suit programmation *ou* réseaux, sans autre condition.

Le même ensemble se simplifie par plusieurs chemins ; l'ordre des lois n'est pas imposé, mais descendre d'abord les complémentaires puis distribuer donne presque toujours le chemin le plus court. En cas d'impasse, repartir de l'expression initiale : les erreurs de simplification se propagent, et un nouveau départ coûte moins que la recherche d'une faute de signe au milieu du calcul.

Exemple résolu 1.10 : passer par les lois de De Morgan

Simplifions $F = \overline{\overline{A} \cup \overline{B}} \cup (A \cap B)$. La première parenthèse se traite par De Morgan :

$$\overline{\overline{A} \cup \overline{B}} = \overline{\overline{A}} \cap \overline{\overline{B}} = A \cap B,$$

en appliquant l'involutivité du complémentaire. L'expression devient $F = (A \cap B) \cup (A \cap B)$, que l'idempotence de $\cup$ réduit à $F = A \cap B$. Le contrôle par Venn est immédiat : la zone extérieure à « A seul $\cup$ B seul » est exactement le lentille central $A \cap B$.

Attention

Une simplification ne se termine que par un contrôle. Le réflexe le plus sûr est le test numérique : affecter aux ensembles A et B des petits ensembles concrets (deux sous-ensembles de $\{1, 2, 3, 4\}$ suffisent), puis calculer l'expression de départ et le résultat proposé avec les deux. Si les deux calculs diffèrent, la simplification est fausse ; s'ils coïncident, elle est très probablement juste, la méthode algébrique garantissant le reste. Sauter ce contrôle revient à confondre une réécriture élégante avec une réécriture correcte.

Ce qu'il faut retenir

Un ensemble se décrit en extension (liste des éléments) ou en compréhension (propriété d'appartenance) ; son cardinal $|A|$ compte ses éléments. Les trois opérations (union $A \cup B$, intersection $A \cap B$, complémentaire $\overline{A}$) se lisent sur le diagramme de Venn et s'ordonnent par les lois du tableau 1.3, dont les deux lois de De Morgan sont les plus rentables : $\overline{A \cup B} = \overline{A} \cap \overline{B}$ et $\overline{A \cap B} = \overline{A} \cup \overline{B}$. Pour compter : $|A \cup B| = |A| + |B| - |A \cap B|$, un ensemble à n éléments possède 2^n sous-ensembles, et $|A \times B| = |A| \times |B|$.

Chapitre 2

Logique et algèbre de Boole

En quelques lignes

Chaque instruction d'un programme repose sur un test, chaque circuit d'un processeur sur un assemblage de portes, et l'un comme l'autre se décrivent avec les mêmes outils : les propositions, leurs tables de vérité et l'algèbre de Boole. Ce chapitre introduit les opérations logiques, étudie les énoncés conditionnels et biconditionnels, puis applique les connecteurs bit à bit aux mots machine. Il se termine par les portes logiques et les circuits, où le calcul booléen devient électronique. La logique des propositions demande aucune notion des chapitres précédents ; les opérations bit à bit en réutilisent l'écriture binaire.

1 Opérations logiques

Définition 2.1 : proposition

Une *proposition* est un énoncé déclaratif qui est soit vrai, soit faux, mais jamais les deux à la fois. La valeur de vérité d'une proposition se note 1 lorsqu'elle est vraie et 0 lorsqu'elle est fausse ; une lettre comme P ou Q désigne une proposition indéterminée, appelée variable propositionnelle.

Les phrases « Lomé est la capitale du Togo » (vraie), « 12 est divisible par 5 » (fausse) et « $x > 0$ » tant que x n'est pas fixé : les deux premières sont des propositions, la troisième ne le devient qu'après avoir donné une valeur à x. Une question (« Quel est le plus grand diviseur de 16 ? »), une exclamation ou une ordre (« Fermez la session ! ») ne sont pas des propositions, car elles ne peuvent recevoir ni le vrai ni le faux.

Définition 2.2 : table de vérité

La *table de vérité* d'une formule logique énumère toutes les attributions de valeurs possibles à ses lettres et donne, pour chacune, la valeur de la formule. Une formule à n lettres a une table de 2^n lignes, chaque lettre valant 0 ou 1 indépendamment des autres.

Les connecteurs construisent des propositions composées à partir de propositions simples. La *négation* $\neg P$ (« non P ») est vraie lorsque P est fausse. La *conjonction* $P \wedge Q$ (« P et Q ») n'est vraie que si les deux le sont. La *disjonction* $P \vee Q$ (« P ou Q ») est vraie dès que l'une l'est : le « ou » de l'informatique est inclusif. Le *ou exclusif* $P \veebar Q$ (« P ou bien Q ») est vrai lorsque exactement l'une des deux l'est, comme le choix entre deux modes de paiement exclusifs. Le tableau 2.1 fixe les valeurs des six connecteurs de base, dont l'implication $\to$ et l'équivalence $\leftrightarrow$ étudiées à la section suivante.

P	Q	$\neg P$	$P \wedge Q$	$P \vee Q$	$P \veebar Q$	$P \to Q$	$P \leftrightarrow Q$
1	1	0	1	1	0	1	1
1	0	0	0	1	1	0	0
0	1	1	0	1	1	1	0
0	0	1	0	0	0	1	1

Table 2.1 – Tables de vérité des six connecteurs usuels

Exemple résolu 2.1 : reconnaître et évaluer des propositions

Parmi les énoncés suivants, seuls les trois premiers sont des propositions : « le bit 1 représente une tension haute » (vraie), « $2^{10} = 1000$ » (fausse, car $2^{10} = 1024$) et « si le fichier est corrompu, alors la sauvegarde restaure les données » (composée, dont la valeur dépend du fichier et de la sauvegarde). L'interrogative « le fichier est-il corrompu ? » et l'injonction « exécute le programme » n'en sont pas. Évaluons ensuite la formule $P \wedge \neg Q$ avec P : « le disque est plein » (vraie) et Q : « la corbeille est vide » (fausse). On a $\neg Q$ vraie, donc $P \wedge \neg Q$ est vraie : le disque est plein et la corbeille n'est pas vide.

Exemple résolu 2.2 : table complète d'une formule à trois lettres

Construisons la table de $(P \vee Q) \wedge R$. La formule a trois lettres : sa table compte $2^3 = 8$ lignes, énumérées dans un ordre fixe, comme si l'on comptait de 0 à 7 en binaire. Les colonnes intermédiaires $P \vee Q$, puis R, puis la formule complète se calculent ligne par ligne.

P	Q	R	$P \vee Q$	R	$(P \vee Q) \wedge R$
1	1	1	1	1	1
1	1	0	1	0	0
1	0	1	1	1	1
1	0	0	1	0	0
0	1	1	1	1	1
0	1	0	1	0	0
0	0	1	0	1	0
0	0	0	0	0	0

Table 2.2 – Table à $2^3 = 8$ lignes de la formule $(P \vee Q) \wedge R$

La lecture est immédiate : la formule vaut 1 exactement sur les trois lignes où R vaut 1 *et* où l'un au moins de P, Q vaut 1. Le nombre de lignes double à chaque lettre ajoutée : quatre lettres donnent 16 lignes, dix lettres donnent 1024 lignes : c'est pourquoi les méthodes algébriques de la section 6 deviennent indispensables.

2 Énoncés conditionnels et biconditionnels

Définition 2.3 : implication

L'*implication* $P \to Q$, qui se lit « si P alors Q », est fausse uniquement dans le cas où P est vraie et Q fausse. P en est l'*hypothèse*, Q la *conclusion*. Lorsque l'hypothèse est fausse, l'implication est vraie : « si un octet vaut plus que 255, alors le réseau est saturé » ne peut être contredite par aucun octet réel.

À toute implication on attache deux autres implications. La *réciproque* de $P \to Q$ est $Q \to P$, qui échange hypothèse et conclusion ; la *contraposée* est $\neg Q \to \neg P$, qui garde l'ordre mais nie les deux membres. Prenons P : « la figure est un carré » et Q : « la figure a quatre côtés ». L'implication $P \to Q$ est vraie ; sa réciproque est fausse, car un rectangle a quatre côtés sans être un carré ; sa contraposée « si la figure n'a pas quatre côtés, alors ce n'est pas un carré » est vraie. Le tableau 2.3 montre que la contraposée a exactement la même table que l'implication, alors que la réciproque s'en distingue :

P	Q	$P \to Q$	$Q \to P$	$\neg Q \to \neg P$
1	1	1	1	1
1	0	0	1	0
0	1	1	0	1
0	0	1	1	1

Table 2.3 – Implication, réciproque et contraposée : les colonnes 3 et 5 coïncident

Définition 2.4 : biconditionnel

L'*équivalence* (ou biconditionnel) $P \leftrightarrow Q$, qui se lit « P si et seulement si Q », est vraie lorsque P et Q ont la même valeur de vérité. Elle exprime que les deux implications se tiennent à la fois : $P \leftrightarrow Q \equiv (P \to Q) \wedge (Q \to P)$.

Dire « un fichier s'ouvre si et seulement s'il n'est pas corrompu », c'est affirmer ensemble les deux sens : un fichier non corrompu s'ouvre, et un fichier qui s'ouvre n'est pas corrompu. Il suffit qu'un seul des deux sens tombe pour que le biconditionnel tombe.

Exemple résolu 2.3 : le raisonnement de l'accusé

Un raisonnement classique rapporte l'argument suivant : « l'accusé n'a pu se rendre coupable que s'il était à New York à 18 heures le 1er janvier ; or il a été établi qu'il était à ce moment-là à Washington ; il n'est donc pas coupable ». Posons C : « l'accusé est coupable » et N : « il était à New York à 18 heures le 1er janvier ». La première phrase se traduit par $C \to N$, l'observation par $\neg N$ (être à Washington exclut d'être à New York à cet instant), et la conclusion par $\neg C$. Le raisonnement entier s'écrit $\big((C \to N) \wedge \neg N\big) \to \neg C$, et sa table à $2^2 = 4$ lignes se calcule colonne par colonne :

C	N	$C \to N$	$\neg N$	$(C \to N) \wedge \neg N$	$\neg C$	formule complète
1	1	1	0	0	0	1
1	0	0	1	0	0	1
0	1	1	0	0	1	1
0	0	1	1	1	1	1

Table 2.4 – Le raisonnement de l'accusé : la dernière colonne ne contient que des 1

La dernière colonne ne contient que des 1 : le raisonnement est *valide*, quelles que soient les vérités effectives de C et de N. La logique ne dit pas si l'accusé est coupable ; elle garantit que, si les prémisses tiennent, la conclusion s'impose.

Exemple résolu 2.4 : de la phrase à la formule

Traduisons deux énoncés d'un règlement de salle informatique. Première phrase : « si l'étudiant a payé sa contribution, alors il accède au réseau ». Posons P : « l'étudiant a payé sa contribution » et Q : « il accède au réseau » ; la phrase se traduit par $P \to Q$, dont la réciproque $Q \to P$ (« s'il accède au réseau, c'est qu'il a payé ») n'est pas automatiquement vraie, l'accès pouvant exister par d'autres voies. Seconde phrase : « l'étudiant imprime si et seulement si son compte est crédité » ; posons I : « l'étudiant imprime » et C : « son compte est crédité » ; elle se traduit par $I \leftrightarrow C$, c'est-à-dire $(I \to C) \wedge (C \to I)$: sans crédit, pas d'impression, et toute impression prouve un crédit.

En programmation, l'implication se matérialise par l'instruction conditionnelle « SI … ALORS … FINSI » (dite « IF … THEN » en anglais). Le fragment qui lit un nombre et le multiplie par 2 lorsqu'il est positif et impair s'écrit :

```
LIRE x   SI x > 0 ET x impair ALORS   y = 2 * x   FINSI
```

La condition du SI est elle-même une proposition composée : « $x > 0$ » et « x est impair » sont reliées par un ET logique, exactement au sens du tableau 2.1. La machine n'exécute le bloc ALORS que lorsque cette condition vaut 1 ; les tests d'un programme se conçoivent et se déboguent ainsi avec les mêmes tables de vérité que le reste du chapitre.

3 Les opérations bit à bit

Un processeur applique les connecteurs non plus à des propositions, mais directement aux bits de deux mots de même longueur : chaque position est traitée indépendamment des autres, c'est l'*opération bit à bit*. Ces opérations servent à forcer, tester ou masquer certains bits d'un mot sans toucher aux autres, ce qui en fait l'outil quotidien de la programmation système.

Exemple résolu 2.5 : évaluation bit à bit de deux mots de huit bits

Soient $P = 01100111$ et $Q = 11110001$ deux mots de huit bits. On aligne les deux mots et l'on calcule position par position.

Mot ou opération	bits, lus de gauche à droite							
P	0	1	1	0	0	1	1	1
Q	1	1	1	1	0	0	0	1
$P \wedge Q$	0	1	1	0	0	0	0	1
$P \vee Q$	1	1	1	1	0	1	1	1
$P \veebar Q$	1	0	0	1	0	1	1	0

Table 2.5 – Opérations bit à bit sur $P = 01100111$ et $Q = 11110001$

Détaillons deux positions. En position 1, P vaut 0 et Q vaut 1 : le produit logique vaut $0 \wedge 1 = 0$, la somme logique $0 \vee 1 = 1$ et l'ou exclusif $0 \veebar 1 = 1$. En position 6, P vaut 1 et Q vaut 0 : on obtient $1 \wedge 0 = 0$, $1 \vee 0 = 1$ et $1 \veebar 0 = 1$. Le tableau donne ainsi $P \wedge Q = 01100001$, $P \vee Q = 11110111$ et $P \veebar Q = 10010110$.
Ces opérations deviennent de vrais outils lorsqu'on les applique avec un *masque*, un mot binaire choisi pour n'agir que sur certaines positions. Effectuer $M \wedge 0000\,1111$ conserve les quatre bits de droite de M et met les autres à zéro : c'est la lecture d'un demi-octet. Effectuer $M \vee 0000\,0001$ force le bit de droite à 1 sans toucher aux autres ; effectuer $M \veebar 0000\,0001$ le bascule, le changeant en son contraire. Tester, forcer, basculer : les trois gestes élémentaires de la programmation système reposent sur ce triplet.

Exemple résolu 2.6 : chiffrement d'un message par le ou exclusif

Le ou exclusif est sa propre réciproque : pour tout bit a, $a \veebar a = 0$ et $a \veebar 0 = a$, si bien que combiner deux fois un mot avec la même clé le restitue : $(M \veebar K) \veebar K = M$. Si M est un message en binaire et K une clé de même longueur, le message chiffré est $C = M \veebar K$, et le destinataire retrouve M en appliquant la même opération avec la même clé. Chiffrons $M = 1011\,0100\,1111\,0001$ avec la clé $K = 1010\,1110\,0100\,1110$:

$$\begin{aligned} M &= \texttt{1011 0100 1111 0001} \\ K &= \texttt{1010 1110 0100 1110} \\ C = M \veebar K &= \texttt{0001 1010 1011 1111} \\ C &= \texttt{0001 1010 1011 1111} \\ K &= \texttt{1010 1110 0100 1110} \\ C \veebar K &= \texttt{1011 0100 1111 0001} = M \end{aligned}$$

Détaillons le premier groupe : $1 \veebar 1 = 0$, $0 \veebar 0 = 0$, $1 \veebar 1 = 0$, $1 \veebar 0 = 1$, d'où 0001 ; les autres groupes se traitent de même. Le déchiffrement par la même clé restitue le message initial : c'est le principe du chiffrement à clé symétrique, où une même clé secrète sert à chiffrer puis à déchiffrer.

Remarque

[le bit de parité] Le ou exclusif fournit aussi un contrôle d'erreur élémentaire. On ajoute à un mot binaire un *bit de parité* choisi pour que le nombre total de 1 soit pair ; le récepteur recompte : si le total devient impair, au moins un bit a changé en route, et le message est

rejeté. Le contrôle repose sur la propriété $a \veebar a = 0$: le ou exclusif de tous les bits du mot correct vaut 0, et toute erreur bascule le résultat à 1. Ce contrôle simple, dit VRC, détecte toute erreur impaire et équipe encore les liaisons série les plus modestes.

4 Circuits et portes logiques

Définition 2.5 : porte logique

Une *porte logique* est un composant électronique élémentaire qui réalise une fonction logique : elle reçoit des états d'entrée codés par des niveaux de tension (0 ou 1) et produit une sortie imposée, pour chaque combinaison d'entrées, par la table de vérité de la fonction réalisée.

Porte	**Nom anglais**	**Expression**	**Sorties pour** $AB = 00, 01, 10, 11$
ET	AND	$S = A \cdot B$	0 0 0 1
OU	OR	$S = A + B$	0 1 1 1
NON	NOT	$S = \overline{A}$ (une entrée)	1 pour $A = 0$; 0 pour $A = 1$
NON-ET	NAND	$S = \overline{A \cdot B}$	1 1 1 0
NON-OU	NOR	$S = \overline{A + B}$	1 0 0 0
OU exclusif	XOR	$S = A \veebar B$	0 1 1 0

Table 2.6 – Les six portes logiques de base, leur expression et leurs sorties

Chaque porte possède un symbole graphique normalisé, reconnaissable à sa silhouette : la porte ET a un côté gauche plat pour les entrées et un côté droit arrondi ; la porte OU a la forme d'un bouclier à pointe arrondie ; la porte NON est un triangle suivi d'un petit cercle d'inversion ; les portes NAND et NOR reprennent les silhouettes du ET et du OU avec le même cercle en sortie. La figure 2.1 montre le circuit analysé dans l'exemple suivant : un inverseur, deux portes ET et une porte OU.

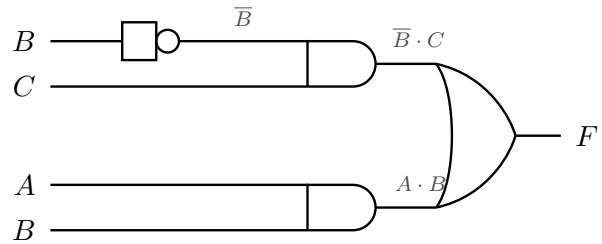


Figure 2.1 – Circuit réalisant $F = A \cdot B + \overline{B} \cdot C$: un inverseur, deux ET et un OU

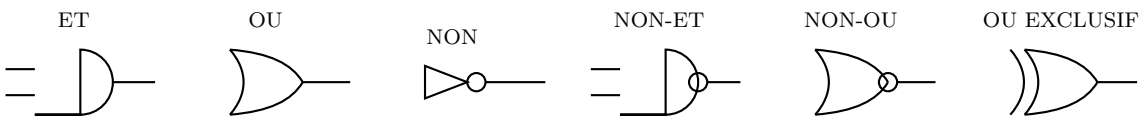


Figure 2.2 – Les six silhouettes de portes : le cercle en sortie marque l'inversion

Exemple résolu 2.7 : analyse du circuit $F = A \cdot B + \overline{B} \cdot C$

Décrivons le circuit de la figure 2.1 porte par porte : le fil B traverse l'inverseur qui produit $\overline{B}$; la porte ET du bas reçoit A et B et produit $A \cdot B$; la porte ET du haut reçoit $\overline{B}$ et C et produit $\overline{B} \cdot C$; la porte OU finale réunit les deux produits et fournit $F = A \cdot B + \overline{B} \cdot C$. La table du circuit se construit en une colonne par fil :

A	B	C	$\overline{B}$	$A \cdot B$	$\overline{B} \cdot C$	F
0	0	0	1	0	0	0
0	0	1	1	0	1	1
0	1	0	0	0	0	0
0	1	1	0	0	0	0
1	0	0	1	0	0	0
1	0	1	1	0	1	1
1	1	0	0	1	0	1
1	1	1	0	1	0	1

Table 2.7 – Table de vérité complète du circuit $F = A \cdot B + \overline{B} \cdot C$

Lisons la ligne $ABC = 101$: $\overline{B} = 1$, $A \cdot B = 1 \cdot 0 = 0$ et $\overline{B} \cdot C = 1 \cdot 1 = 1$, d'où $F = 0 + 1 = 1$. Le tableau montre que F vaut 1 exactement pour les lignes 001, 101, 110 et 111, celles où A et B valent 1 ensemble ou où B vaut 0 pendant que C vaut 1 : le circuit agit comme un aiguillage qui suit $A \cdot B$ lorsque B vaut 1 et $\overline{B} \cdot C$ lorsque B vaut 0.

Exemple résolu 2.8 : concevoir la fonction majoritaire

Un système de vote à trois délégués valide une motion si au moins deux d'entre eux votent pour. Notons A, B, C les trois votes (1 pour, 0 contre) et cherchons la fonction M qui vaut 1 dès que deux entrées au moins valent 1. Les lignes favorables de la table sont 011, 101, 110 et 111 ; on écrit donc chaque ligne favorable comme un produit et l'on somme les produits :

$$M = \overline{A} \cdot B \cdot C + A \cdot \overline{B} \cdot C + A \cdot B \cdot \overline{C} + A \cdot B \cdot C.$$

Chaque produit vaut 1 sur exactement sa ligne, et leur somme couvre les quatre lignes favorables : c'est la *forme canonique* de la fonction, où chaque terme correspond à une ligne de la table. Le circuit se dessine alors mécaniquement : un inverseur par lettre niée, quatre portes ET à trois entrées, une porte OU à quatre entrées. On peut réduire cette première version par l'algèbre de la section 6 : en regroupant les termes deux à deux, M se simplifie en $A \cdot B + A \cdot C + B \cdot C$, trois produits de deux entrées seulement, soit un circuit plus petit pour la même fonction.

5 Propriétés des opérations logiques

Définition 2.6 : équivalence logique

Deux formules A et B sont *logiquement équivalentes*, noté $A \equiv B$, lorsqu'elles ont la même table de vérité : pour toute attribution de valeurs aux lettres, elles prennent la même valeur. Une *tautologie* est une formule vraie sur toutes les lignes de sa table, une *contradiction* une formule fausse sur toutes les lignes.

Propriété 2.1 : équivalences fondamentales

Pour toutes propositions P, Q et R :

- lois de De Morgan : $\neg(P \vee Q) \equiv \neg P \wedge \neg Q$ et $\neg(P \wedge Q) \equiv \neg P \vee \neg Q$;
- distributivités : $P \wedge (Q \vee R) \equiv (P \wedge Q) \vee (P \wedge R)$ et $P \vee (Q \wedge R) \equiv (P \vee Q) \wedge (P \vee R)$;
- implication : $P \to Q \equiv \neg P \vee Q$; contraposée : $P \to Q \equiv \neg Q \to \neg P$;
- négation d'une implication : $\neg(P \to Q) \equiv P \wedge \neg Q$;
- double négation : $\neg\neg P \equiv P$; biconditionnel : $P \leftrightarrow Q \equiv (P \to Q) \wedge (Q \to P)$.

Chaque équivalence se vérifie en dressant les deux tables et en constatant qu'elles coïncident.

Exemple résolu 2.9 : l'implication se ramène au ou et à la négation

Montrons $P \to Q \equiv \neg P \vee Q$ par la table. On calcule les colonnes de $P \to Q$, de $\neg P$ puis de $\neg P \vee Q$:

P	Q	$P \to Q$	$\neg P$	$\neg P \vee Q$
1	1	1	0	1
1	0	0	0	0
0	1	1	1	1
0	0	1	1	1

Table 2.8 – Équivalence $P \to Q \equiv \neg P \vee Q$: les colonnes 3 et 5 coïncident

Les colonnes 3 et 5 sont identiques ligne à ligne : l'équivalence est établie. Ce résultat permet d'exprimer l'implication avec les seuls connecteurs $\vee$ et $\neg$, dont on verra qu'ils suffisent à construire tous les circuits.

Exemple résolu 2.10 : une contradiction et une tautologie

Considérons $F_1 = P \wedge \neg(P \vee Q)$, puis $F_2 = (P \wedge Q) \vee \neg(P \wedge Q)$.

P	Q	$P \vee Q$	$\neg(P \vee Q)$	$P \wedge \neg(P \vee Q)$
1	1	1	0	0
1	0	1	0	0
0	1	1	0	0
0	0	0	1	0

Table 2.9 – $P \wedge \neg(P \vee Q)$ vaut 0 sur les quatre lignes : contradiction

La formule F_1 affirme P et, en même temps, la négation de $P \vee Q$, qui contient P : elle se contredit elle-même et vaut 0 partout, c'est une *contradiction*. La formule F_2, de structure $A \vee \neg A$, affirme une chose ou sa négation sans troisième possibilité : sa table ne contient que des 1, c'est une *tautologie* : c'est le principe du tiers exclu appliqué à la proposition $P \wedge Q$. Toute formule de la forme $A \vee \neg A$ est ainsi une tautologie, et toute formule de la forme $A \wedge \neg A$ une contradiction.

Attention

La négation d'une implication n'est pas l'implication des négations : $\neg(P \to Q) \not\equiv \neg P \to \neg Q$. En français courant, « il est faux que, si j'ai le baccalauréat, je suis inscrit » ne signifie pas « si je n'ai pas le baccalauréat, je ne suis pas inscrit » : l'inscription peut avoir d'autres voies. La vraie négation est $P \wedge \neg Q$ (j'ai le baccalauréat *et* je ne suis pas inscrit), tandis que $\neg P \to \neg Q$ n'est autre que la réciproque de l'implication initiale.

Remarque

[De Morgan dans les programmes] Les lois de De Morgan s'utilisent chaque fois qu'un programmeur écrit la négation d'une condition complexe. La négation de « le fichier existe *et* il est lisible » n'est pas « le fichier n'existe pas et il n'est pas lisible », mais « le fichier n'existe pas *ou* il n'est pas lisible » : $\neg(P \wedge Q) \equiv \neg P \vee \neg Q$. En code, réécrire la condition sous la bonne forme évite des bugs subtils : un « ou » oublié dans une négation inverse complètement la logique du programme.

6 Simplification booléenne

Pour transformer les formules sans redresser des tables toujours plus grandes, George Boole (1815–1864) a proposé de manipuler le raisonnement au moyen d'équations : le vrai et le faux y deviennent les valeurs 1 et 0, le « et » une multiplication et le « ou » une addition. Claude Shannon a montré en 1938 que cette algèbre décrit exactement le comportement des circuits numériques : elle sert depuis en permanence, pour *analyser* un circuit existant en écrivant son équation, et pour *concevoir* la réalisation la plus simple d'une fonction voulue.

Définition 2.7 : algèbre de Boole

L'algèbre de Boole sur $\{0,1\} = \{0,1\}$ comporte deux opérations binaires, la *somme logique* $+$ (le « ou ») et le *produit logique* $\cdot$ (le « et »), et une opération unaire, la *complémentation* notée par une barre. Les postulats fixent le calcul sur 0 et 1.

Produit logique ($\cdot$)	**Somme logique ($+$)**
$0 \cdot 0 = 0$	$0 + 0 = 0$
$0 \cdot 1 = 1 \cdot 0 = 0$	$1 + 0 = 0 + 1 = 1$
$1 \cdot 1 = 1$	$1 + 1 = 1$
$\overline{0} = 1$	$\overline{1} = 0$

Table 2.10 – Les postulats de l'algèbre de Boole sur les éléments 0 et 1

Le seul postulat en désaccord avec l'arithmétique ordinaire est $1 + 1 = 1$: la somme logique n'est pas l'addition binaire du chapitre 3, qui écrit $1+1 = 10$ avec un report. Les règles d'écriture suivent l'usage algébrique : le produit a préséance sur la somme ($a + b \cdot c$ se lit $a + (b \cdot c)$), le point de multiplication s'omet (ab pour $a \cdot b$), et seules les parenthèses forcent un autre ordre. Le tableau 2.11 rassemble les théorèmes usuels, que l'on démontre par les axiomes ou par une table complète.

Théorème	**Forme somme**	**Forme produit**
Élément absorbant	$a + 1 = 1$	$a \cdot 0 = 0$
Complémentation	$a + \overline{a} = 1$	$a \cdot \overline{a} = 0$
Idempotence	$a + a = a$	$a \cdot a = a$
Absorption	$a + a \cdot b = a$	$a \cdot (a + b) = a$
Simplification	$a + \overline{a} \cdot b = a + b$	$a \cdot (\overline{a} + b) = a \cdot b$
Involution	$\overline{\overline{a}} = a$	
Lois de De Morgan	$\overline{a + b} = \overline{a} \cdot \overline{b}$	$\overline{a \cdot b} = \overline{a} + \overline{b}$

Table 2.11 – Théorèmes usuels de l'algèbre de Boole (a, b éléments de $\{0, 1\}$)

À titre de modèle, déroulons la démonstration algébrique de la seconde forme de l'absorption, chaque passage étant justifié par un axiome ou un théorème déjà acquis :

$$\begin{aligned} a \cdot (a + b) &= (a + 0) \cdot (a + b) && \text{élément neutre : } a = a + 0 \\ &= a + 0 \cdot b && \text{distributivité (seconde forme)} \\ &= a + 0 && \text{élément absorbant : } b \cdot 0 = 0 \\ &= a. && \text{élément neutre. CQFD} \end{aligned}$$

Le dual $a + a \cdot b = a$ est acquis sans calcul, par le principe de dualité.

Remarque

[le principe de dualité] Chaque théorème du tableau 2.11 possède un *dual* qui s'obtient en échangeant $+$ et $\cdot$, puis 0 et 1, sans toucher à la barre de complément : de l'idempotence $a+a = a$ on déduit $a \cdot a = a$, de l'absorbant $a+1 = 1$ on déduit $a \cdot 0 = 0$. Dès qu'un résultat est prouvé, son dual est acquis, et les deux colonnes du tableau se mémorisent d'un même mouvement.

Méthode : simplifier une expression booléenne

1. appliquer d'abord De Morgan pour faire descendre les barres sur les variables seules ;
2. distribuer pour faire apparaître les paires $a \cdot \overline{a} = 0$ et $a + \overline{a} = 1$;
3. réduire à l'aide de l'idempotence, de l'absorption et de la simplification $a + \overline{a} \cdot b = a + b$;
4. contrôler en comparant les tables de l'expression initiale et du résultat sur quelques lignes, ou en re-dessinant le circuit réduit.

Exemple résolu 2.11 : simplifications algébriques pas à pas

Simplifions $E_1 = (A + B) \cdot (A + \overline{B})$, puis $E_2 = \overline{A} \cdot B + A \cdot B + \overline{A} \cdot \overline{B}$.

$$\begin{aligned} E_1 &= A + B \cdot \overline{B} && \text{distributivité : } (a+b)(a+c) = a + bc \\ &= A + 0 && \text{complémentation} \\ &= A. && \text{élément neutre} \end{aligned}$$

Un circuit qui réalisait trois portes pour E_1 se réduit à un simple fil portant le signal A. Pour E_2, on groupe les deux premiers produits :

$$\begin{aligned} E_2 &= (\overline{A} + A) \cdot B + \overline{A} \cdot \overline{B} && \text{distributivité} \\ &= 1 \cdot B + \overline{A} \cdot \overline{B} && \text{complémentation} \\ &= B + \overline{A} \cdot \overline{B} && \text{élément neutre} \\ &= (B + \overline{A}) \cdot (B + \overline{B}) && \text{distributivité (seconde forme)} \\ &= (B + \overline{A}) \cdot 1 && \text{complémentation} \\ &= B + \overline{A}. && \text{élément neutre} \end{aligned}$$

L'expression initiale demandait trois ET, deux OU et deux inverseurs ; le résultat $B + \overline{A}$ s'obtient avec deux inverseurs et un OU. La fonction est inchangée, le circuit est plus petit, plus rapide et moins coûteux : c'est tout l'intérêt de la simplification booléenne.

Exemple résolu 2.12 : faire descendre une barre avec De Morgan

Simplifions $F = \overline{\overline{A} + B} + A \cdot B$. La grande barre se traite d'abord par De Morgan :

$$\overline{\overline{A} + B} = \overline{\overline{A}} \cdot \overline{B} = A \cdot \overline{B},$$

en appliquant l'involution au complément double. Il reste $F = A \cdot \overline{B} + A \cdot B$, et la distributivité donne $F = A \cdot (\overline{B} + B) = A \cdot 1 = A$. Le contrôle par la table est immédiat : F vaut 1 exactement lorsque A vaut 1, quelle que soit la valeur de B, ce que l'expression initiale réalisait par un détour plus coûteux.

Ce qu'il faut retenir

Une proposition est vraie ou fausse ; une formule à n lettres a une table de 2^n lignes, et les six connecteurs usuels $\neg$, $\wedge$, $\vee$, $\veebar$, $\to$, $\leftrightarrow$ ont leurs tables fixes (tableau 2.1). L'implication n'est fausse qu'en cas $1 \to 0$, sa contraposée $\neg Q \to \neg P$ a sa table, sa réciproque non ; le

biconditionnel exprime la double implication. Les opérations bit à bit appliquent $\wedge$, $\vee$, $\veebar$ position par position, et le ou exclusif, propre inverse de lui-même, fonde le chiffrement à clé symétrique. En algèbre de Boole, $1 + 1 = 1$, et les théorèmes du tableau 2.11 (absorption, simplification, De Morgan) réduisent expressions et circuits. Toute cette panoplie se travaille à la séance de travaux dirigés 2, de la table complète au circuit réduit.

Chapitre 3

Systèmes de numération et opérations

En quelques lignes

Une machine ne connaît que deux états stables (le courant passe ou ne passe pas), auxquels on associe les valeurs 0 et 1 : tout nombre, tout texte, toute image finit par se réduire à des suites de bits. Ce chapitre définit les systèmes de numération utilisés en informatique (binaire, octal, décimal, hexadécimal), apprend à convertir un nombre d'une base vers l'autre dans les deux sens, pose les opérations arithmétiques directement en binaire et en hexadécimal, puis mesure l'information elle-même en bits et en octets. Aucune notion d'informatique n'est supposée connue ; les puissances et la division euclidienne suffisent.

1 Systèmes de numération

Définition 3.1 : système de numération

Un *système de numération* de base b (entier $b \geqslant 2$) est défini par deux éléments : la *base* b, qui fixe le nombre de symboles disponibles, et l'ensemble des b *symboles* (ou chiffres) autorisés. Tout nombre s'écrit comme une suite finie de ces symboles, et chacun d'eux vaut un entier strictement inférieur à b.

Système	Base	Symboles	Nombre de symboles
Décimal	10	$0, 1, 2, 3, 4, 5, 6, 7, 8, 9$	10
Binaire	2	$0, 1$	2
Octal	8	$0, 1, 2, 3, 4, 5, 6, 7$	8
Hexadécimal	16	$0, \ldots, 9$, puis A, B, C, D, E, F	16

Table 3.1 – Les quatre systèmes de numération de l'informatique

Les six lettres de l'hexadécimal ne sont pas des variables : elles valent $A = 10$, $B = 11$, $C = 12$, $D = 13$, $E = 14$ et $F = 15$. Ce recours aux lettres s'impose parce qu'un chiffre doit rester un caractère unique : écrire « 10 » pour dix créerait une ambiguïté avec le nombre dix écrit sur deux chiffres. L'hexadécimal existe pour une raison pratique : un nombre binaire occupe beaucoup de chiffres, et l'hexadécimal, étudié à la section 2, le condense par groupes de quatre bits.

L'humanité a pratiqué plusieurs bases avant celle-ci : la base 60 des Mésopotamiens, dont il nous reste les minutes et les secondes ; la base 20 des anciens Gaulois, qui survit dans « quatre-vingts » ; la base 12 des douzaines d'œufs et des mois de l'année. La base 10 s'est imposée dans la vie courante parce que nous comptons sur nos dix doigts ; l'informatique, elle, impose la base 2 pour une raison physique expliquée ci-dessous, et lui adjoint les bases 8 et 16 comme notations compactes.

Définition 3.2 : notation positionnelle, bit, poids

Soit $N = (a_{n-1}\, a_{n-2}\, \ldots\, a_1\, a_0)_b$ une écriture en base b : n est le nombre de symboles employés, a_i le symbole de position i, chaque a_i valant un entier strictement inférieur à b, et les positions se comptent de droite à gauche en commençant à 0. En binaire, chaque symbole 0 ou 1 s'appelle un *bit* (contraction de l'anglais *binary digit*) ; le bit le plus à gauche, de plus fort poids, s'appelle *bit de poids fort*, et celui de droite *bit de poids faible*.

Exemple résolu 3.1 : lire les positions et repérer une écriture invalide

Dans $(19017)_{10}$, on lit $a_4 = 1$, $a_3 = 9$, $a_2 = 0$, $a_1 = 1$ et $a_0 = 7$. Dans $(1011101)_2$, mot de sept bits, $a_6 = 1$ est le bit de poids fort et $a_0 = 1$ le bit de poids faible. En revanche l'écriture $(18095)_8$ *n'existe pas* : en base 8, les symboles autorisés sont 0 à 7, or le chiffre 8 et le chiffre 9 y figurent. Avant toute conversion, vérifier que chaque symbole est strictement inférieur à la base.

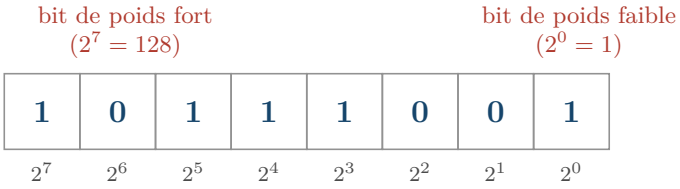


Figure 3.1 – Le mot $(10111001)_2$: chaque position porte le poids 2^i

Exemple résolu 3.2 : décomposer un octet par la table des poids

Décomposons $(10111001)_2$, un mot de huit bits. Les positions vont de 7 à 0 ; on associe à chaque bit sa valeur, produit du chiffre par son poids.

Position i	7	6	5	4	3	2	1	0
Chiffre a_i	1	0	1	1	1	0	0	1
Valeur $a_i \times 2^i$	128	0	32	16	8	0	0	1

Table 3.2 – Table des poids du nombre $(10111001)_2$

On additionne les valeurs non nulles : $128+32+16+8+1 = 185$. Le résultat est $(10111001)_2 = (185)_{10}$. C'est exactement le calcul que fait le processeur lorsqu'il interprète un code binaire comme une valeur numérique.

Reste à comprendre pourquoi la machine ne connaît que 0 et 1 : un circuit électronique numérique est bâti sur des éléments à deux états stables (tension basse ou haute), et le processeur n'interprète aucun état intermédiaire. Le choix de deux états n'est pas une pauvreté, c'est une robustesse : entre « tension basse » et « tension haute », une marge large sépare les deux valeurs reconnues, si bien qu'un léger bruit électrique ne change pas l'état lu. Dix niveaux de tension distincts, eux, se confondraient dès que le bruit s'en mêle. Le binaire occupe donc une place à part, les bases 8 et 16 ne servant qu'à l'écrire de façon compacte ; la mesure de l'information en bits et en multiples de l'octet fait l'objet de la section 4.

2 Conversions

2.1 D'une base quelconque vers le décimal

Méthode : somme pondérée

Le poids d'un chiffre de position i est la base élevée à la puissance de sa position. La valeur décimale s'obtient en additionnant les produits de chaque chiffre par son poids :

$$(N)_b \;=\; a_{n-1} \times b^{n-1} + a_{n-2} \times b^{n-2} + \cdots + a_1 \times b + a_0.$$

Pour un nombre à partie fractionnaire, les positions à droite de la virgule reçoivent les exposants négatifs $b^{-1}, b^{-2}, \ldots$

Exemple résolu 3.3 : trois conversions vers le décimal

Convertissons $(1023)_4$, $(175)_8$ et $(A24)_{16}$.

$$(1023)_4 = 1 \times 4^3 + 0 \times 4^2 + 2 \times 4^1 + 3 \times 4^0 = 64 + 0 + 8 + 3 = 75.$$
$$(175)_8 = 1 \times 8^2 + 7 \times 8^1 + 5 \times 8^0 = 64 + 56 + 5 = 125.$$
$$(A24)_{16} = 10 \times 16^2 + 2 \times 16^1 + 4 \times 16^0 = 2560 + 32 + 4 = 2596.$$

Pour $(A24)_{16}$, on a d'abord remplacé la lettre A par sa valeur 10. Les seules puissances à connaître par cœur sont, pour le binaire, $1, 2, 4, 8, 16, 32, 64, 128, 256$, pour l'octal $1, 8, 64$ et pour l'hexadécimal $1, 16, 256$: la conversion se fait alors presque de tête, en pointant chaque chiffre non nul et en additionnant les poids.

2.2 Du décimal vers la base b

Méthode : divisions successives

Pour écrire un entier N en base b : diviser N par b et noter le reste ; diviser le quotient par b, noter le reste, et recommencer tant que le quotient n'est pas nul ; lire les restes du **dernier** au **premier**. Le premier reste obtenu est le chiffre des unités, donc il se place à droite du résultat.

Exemple résolu 3.4 : du décimal vers le binaire et vers l'hexadécimal

Convertissons 35 en base 2 :

$$35 = 2 \times 17 + 1 \qquad 17 = 2 \times 8 + 1 \qquad 8 = 2 \times 4 + 0$$
$$4 = 2 \times 2 + 0 \qquad 2 = 2 \times 1 + 0 \qquad 1 = 2 \times 0 + 1$$

La lecture des restes du dernier au premier donne $(35)_{10} = (100011)_2$, que l'on vérifie par la somme pondérée : $32 + 2 + 1 = 35$. Convertissons ensuite 2596 en base 16 : les divisions successives donnent $2596 = 16 \times 162 + 4$, puis $162 = 16 \times 10 + 2$, puis $10 = 16 \times 0 + 10$. Les restes, lus du dernier au premier, valent 10, 2 et 4 ; le reste 10 s'écrivant A, on obtient $(2596)_{10} = (A24)_{16}$, en accord avec la conversion directe de l'exemple précédent.

Attention

L'erreur classique consiste à lire les restes dans l'ordre où ils ont été calculés. Relire les restes de 35 à l'endroit donne 110001, qui vaut $32+16+1=49$ et non 35 : l'écriture inversée désigne un tout autre nombre. Relire du dernier au premier reste est une discipline non négociable.

Les nombres à virgule se traitent par deux méthodes symétriques. Vers le décimal, la somme pondérée se poursuit à droite de la virgule avec les exposants négatifs. Du décimal vers la base b, on multiplie successivement la partie fractionnaire par b : la partie entière de chaque produit donne, de gauche à droite, les chiffres après la virgule, et l'on s'arrête lorsque la partie fractionnaire devient nulle.

Exemple résolu 3.5 : un nombre à virgule dans les deux sens

Convertissons 0,625 en binaire par multiplications successives :

$$\begin{aligned} 0{,}625 \times 2 &= 1{,}25 && \text{partie entière } 1 \\ 0{,}25 \times 2 &= 0{,}5 && \text{partie entière } 0 \\ 0{,}5 \times 2 &= 1{,}0 && \text{partie entière } 1 \end{aligned}$$

La partie fractionnaire vaut 1,0 : le calcul s'arrête, et les parties entières lues de haut en bas donnent $(0{,}101)_2$. Relecture par la somme pondérée : $1\times2^{-1}+0\times2^{-2}+1\times2^{-3}=0{,}5+0{,}25=0{,}625$. Dans l'autre sens, convertissons $(101{,}01)_2$: la partie entière 101 vaut $4+1=5$ et la partie fractionnaire 01 vaut $0\times2^{-1}+1\times2^{-2}=0{,}25$, d'où $(101{,}01)_2=(5{,}25)_{10}$. On notera que certains nombres, comme 0,1, n'ont pas d'écriture binaire finie : la multiplication par 2 ne s'arrête jamais. Ce fait, sans conséquence sur papier, devient la source des erreurs d'arrondi étudiées au chapitre 4.

Remarque

[la méthode de Horner] La somme pondérée se réorganise en un schéma sans puissances : on part du chiffre le plus à gauche, on multiplie par la base, on ajoute le chiffre suivant, et l'on recommence jusqu'au dernier. Pour $(1101)_2$: $\big(((1\times2)+1)\times2+0\big)\times2+1=(3\times2+0)\times2+1=6\times2+1=13$. Cette variante systématique se programme aisément, puisqu'elle ne demande qu'une boucle « multiplier par la base, ajouter le chiffre suivant ».

2.3 Les raccourcis binaire-octal et binaire-hexadécimal

Propriété 3.1 : correspondances par groupes de bits

Puisque $8=2^3$, un chiffre octal correspond exactement à un groupe de trois bits ; puisque $16=2^4$, un chiffre hexadécimal correspond exactement à un groupe de quatre bits. Les conversions se font par simple substitution, sans aucune arithmétique, en regroupant les bits à partir de la droite et en complétant par des zéros à gauche au besoin.

Chiffre octal	Écriture sur 3 bits
0	000
1	001
2	010
3	011
4	100
5	101
6	110
7	111

Table 3.3 – Correspondance chiffres octaux et groupes de trois bits

Chiffre hexa	4 bits	Chiffre hexa	4 bits
0	0000	8	1000
1	0001	9	1001
2	0010	A	1010
3	0011	B	1011
4	0100	C	1100
5	0101	D	1101
6	0110	E	1110
7	0111	F	1111

Table 3.4 – Correspondance chiffres hexadécimaux et groupes de quatre bits

Exemple résolu 3.6 : convertir par substitution dans les deux sens

Convertons $(1\,111\,010\,110\,110)_2$ en octal : en regroupant par trois à partir de la droite, 1 devient 1, puis 111, 010, 110, 110 donnent 7, 2, 6, 6, d'où $(17266)_8$. Convertissons ensuite $(A24)_{16}$ en binaire : chaque chiffre hexadécimal se déplie sur quatre bits, A donne 1010, 2 donne 0010 et 4 donne 0100, d'où $(A24)_{16} = (1010\,0010\,0100)_2$. Ces deux tables sont les seules à connaître par cœur dans tout le chapitre : elles font l'économie de toutes les divisions.

La table suivante donne les écritures des seize premières valeurs dans les quatre systèmes : elle sert de table de correspondance rapide pour toutes les conversions de ce chapitre.

Déc.	Binaire	Octal	Hexa	Déc.	Binaire	Octal	Hexa
0	0000	0	0	8	1000	10	8
1	0001	1	1	9	1001	11	9
2	0010	2	2	10	1010	12	A
3	0011	3	3	11	1011	13	B
4	0100	4	4	12	1100	14	C
5	0101	5	5	13	1101	15	D
6	0110	6	6	14	1110	16	E
7	0111	7	7	15	1111	17	F

Table 3.5 – Correspondance des seize premières valeurs dans les quatre systèmes

3 Opérations dans les systèmes

Les quatre opérations se posent dans toute base exactement comme en décimal : on traite les chiffres de droite à gauche, en tenant compte des retenues lorsque la somme des chiffres atteint la base. En binaire, la table d'addition se résume à quatre cas : $0+0=0$, $0+1=1$, $1+0=1$ et $1+1=10$: j'écris 0 et je reporte 1.

Les tables des opérations binaires tiennent en deux lignes : pour l'addition, $0+0=0$, $0+1=1+0=1$ et $1+1=10$ (j'écris 0, je reporte 1) ; pour la multiplication, $0\times 0=0\times 1=1\times 0=0$ et $1\times 1=1$. C'est toute l'arithmétique de la machine.

Méthode : poser une opération dans une base b

1. écrire les deux nombres alignés à droite, dans la même base, et vérifier que tous leurs chiffres sont strictement inférieurs à b ;
2. traiter les colonnes de droite à gauche, en additionnant (ou multipliant) les chiffres *en décimal* ;
3. si le total atteint b, écrire le reste de la division par b et reporter le quotient ;
4. contrôler le résultat en convertissant les trois nombres en décimal : l'égalité posée doit se vérifier de part et d'autre.

Exemple résolu 3.7 : addition et multiplication binaires posées

Posons $1011+1101$ puis 1011×101, soit en décimal $11+13=24$ et $11\times 5=55$:

```
  1011          1011
 +1101        × 101
 ------       ------
  11000         1011     produit partiel (×1)
                0000     produit partiel (×0, décalé)
               1011      produit partiel (×1, décalé)
              ------
              110111     32 + 16 + 4 + 2 + 1 = 55
```

Dans l'addition, la troisième colonne cumule $1+1+1$ (le report inclus) : j'écris 1 et je reporte 1, d'où le résultat $11000=24$. Dans la multiplication, chaque chiffre du multiplicateur valant

0 ou 1, les produits partiels sont le multiplicande lui-même, décalé, ou la ligne nulle : il ne reste qu'à les additionner.

Exemple résolu 3.8 : soustraction avec emprunt et addition hexadécimale

Posons $1101 - 0110$ en binaire ($13 - 6 = 7$). Dans la colonne des unités, $1 - 0 = 1$. Dans la suivante, $0 - 1$ est impossible : j'emprunte 1 à la colonne de gauche, ce qui donne $10 - 1 = 1$, et la colonne prêteuse passe de 1 à 0 ; dans cette colonne, $0 - 1$ devient à son tour impossible, d'où un second emprunt qui fournit encore $10 - 1 = 1$. Le bit de poids fort, vidé par ce second emprunt, termine le calcul par $0 - 0 = 0$. Le résultat est $(111)_2 = 7$. En hexadécimal, posons $3A7 + 1B9$: colonne des unités, $7 + 9 = 16 = 1 \times 16 + 0$, j'écris 0 et je reporte 1 ; colonne suivante, $A + B + 1 = 10 + 11 + 1 = 22 = 1 \times 16 + 6$, j'écris 6 et je reporte 1 ; dernière colonne, $3 + 1 + 1 = 5$. Le résultat est $(560)_{16}$, que l'on vérifie en décimal : $935 + 441 = 1376 = 5 \times 256 + 6 \times 16 = 1376$.

Remarque

Les nombres négatifs ne s'écrivent pas avec un simple signe dans la machine : ils utilisent la représentation par *complément à deux*, étudiée au chapitre 4, qui permet d'effectuer la soustraction avec le même circuit que l'addition. La soustraction binaire posée ci-dessus reste néanmoins la méthode de calcul manuel à maîtriser. Un cas particulier de la multiplication mérite d'être signalé : multiplier par 2 revient à décaler tous les bits d'une position vers la gauche, $10110 \times 10_2 = 101100$, soit $22 \times 2 = 44$, exactement comme multiplier par 10 en décimal ajoute un zéro.

4 Codes binaires

Propriété 3.2 : nombre de codages sur n bits

Un mot de n bits offre 2 choix par position, donc 2^n codages possibles : n bits codent 2^n valeurs distinctes. Sur 4 bits on code $2^4 = 16$ valeurs (de 0 à 15), sur 8 bits $2^8 = 256$ valeurs (de 0 à 255).

Cette propriété gouverne tous les codages de l'informatique. Un *codage de caractères* associe à chaque lettre, chiffre ou signe un nombre binaire fixé par convention. La table *ASCII* (American Standard Code for Information Interchange), codée historiquement sur 7 bits puis stockée sur 8, attribue par exemple 65 à la lettre A, 66 à B, jusqu'à 90 pour Z, les minuscules occupant 97 à 122, les chiffres 48 à 57 et l'espace 32.

Caractères	Codes décimaux	Exemple
Espace	32	$32 = (0010\,0000)_2$
Chiffres 0 à 9	48 à 57	« 0 » vaut $48 = (0011\,0000)_2$
Lettres A à Z	65 à 90	« C » vaut $67 = (0100\,0011)_2$
Lettres a à z	97 à 122	« e » vaut $101 = (0110\,0101)_2$

Table 3.6 – Extraits de la table ASCII : chaque caractère est un nombre

Exemple résolu 3.9 : coder un mot et compter les octets

Codons le mot « CPTEC » en ASCII : C vaut 67, P vaut 80, T vaut 84, E vaut 69 et C vaut 67 ; stocké sur un octet par caractère, le mot occupe 5 octets, soit $5 \times 8 = 40$ bits, écrits par exemple 0100 0011 0101 0000 0101 0100 0100 0101 0100 0011. La convention « un octet par caractère » limite le jeu à 256 symboles : Un mot accentué comme « é » montre la limite : l'ASCII d'origine, américain, ne contient ni les accents ni les caractères africains, et chaque norme nationale y avait greffé ses propres extensions incompatibles. c'est la raison d'être d'Unicode, qui associe à chaque caractère du monde un numéro universel, stocké sur un, deux, trois ou quatre octets selon la norme d'encodage.

Puissance de 2	Valeur	Unité	Valeur (fabricant)
2^8	256	1 Ko	10^3 octets
2^{10}	1 024	1 Mo	10^6 octets
2^{16}	65 536	1 Go	10^9 octets
2^{20}	1 048 576	1 To	10^{12} octets
2^{30}	1 073 741 824		

Table 3.7 – Puissances de 2 à connaître et multiples décimaux de l'octet

Exemple résolu 3.10 : mesurer l'information : octets et multiples

Un *octet* (*byte* en anglais) est un groupe de 8 bits : c'est l'unité usuelle de l'information. Les multiples du fabricant suivent les puissances de 10 : 1 kilooctet (Ko) $= 10^3$ octets, 1 mégaoctet (Mo) $= 10^6$ octets, 1 gigaoctet (Go) $= 10^9$ octets ; la mémoire vive, elle, se compte en puissances de 2, où $2^{10} = 1024$ octets, $2^{20} = 1\,048\,576$ octets et $2^{30} = 1\,073\,741\,824$ octets. Une photographie de smartphone pèse environ 3 Mo ; sur une clé de 64 Go, soit 64×10^9 octets, on peut en stocker environ

$$\frac{64 \times 10^9}{3 \times 10^6} \approx 21\,000,$$

c'est-à-dire plus de vingt mille photographies, à condition de négliger l'espace réservé par le système de fichiers.

Ce qu'il faut retenir

Un système de numération se définit par sa base et ses symboles ; en informatique : binaire (2), octal (8), décimal (10), hexadécimal (16, avec A=10 à F=15). Vers le décimal : somme pondérée, chaque chiffre multiplié par b^{position}, positions comptées depuis 0 à droite. Du décimal vers la base b : divisions successives par b, restes lus du dernier au premier. Raccourcis : $8 = 2^3$ et $16 = 2^4$, donc trois bits par chiffre octal, quatre bits par chiffre hexadécimal, par simple substitution. Les opérations se posent comme en décimal avec retenue quand la somme atteint la base ; en binaire $1 + 1 = 10$. Enfin n bits codent 2^n valeurs, un octet vaut 8 bits, 1 Ko $= 10^3$ octets chez les fabricants contre $2^{10} = 1024$ pour la mémoire. Ces conversions composent l'essentiel de la séance de travaux dirigés 1.

Chapitre 4

Données en mémoire

En quelques lignes

Après avoir écrit les nombres en binaire, il faut voir où ils habitent : la mémoire d'un ordinateur est une suite d'octets, chacun identifié par une adresse, et tout ce qu'un programme manipule (compteurs, salaires, coordonnées d'un pixel) y prend une place exactement comptée. Ce chapitre présente la représentation des nombres entiers signés par le complément à deux, décrit l'organisation et les capacités de la mémoire, examine les limites de précision qui guettent tout calcul machine et donne les critères du choix d'une représentation adaptée. Les chapitres 1 et 3 fournissent les bases de numération utilisées dans chaque conversion.

1 Représentation des nombres en mémoire

Définition 4.1 : mot mémoire

La mémoire d'une machine stocke l'information par *mots* de taille fixe : 8 bits (un octet), 16, 32 ou 64 bits selon le type de donnée. Sur n bits, un entier *non signé* occupe toutes les valeurs de 0 à $2^n - 1$; pour coder aussi les nombres négatifs, on réserve un codage particulier, le *complément à deux*.

Méthode : coder un entier relatif par complément à deux sur n bits

1. écrire la valeur absolue du nombre en binaire sur n bits ;
2. inverser tous les bits (complément à un) ;
3. ajouter 1 au résultat : c'est le complément à deux, qui code le nombre négatif.

Pour relire un mot signé : si le bit de poids fort vaut 0, le mot se lit directement ; s'il vaut 1, le nombre vaut la valeur binaire du mot moins 2^n.

Une solution plus intuitive existe, dite *grandeur signée* (réserver un bit de signe et coder la valeur absolue sur les bits restants), mais elle a deux défauts : elle donne deux zéros distincts (0000 0000 et 1000 0000 représentent tous deux zéro) et elle oblige à soustraire, alors que le complément à deux permet d'additionner sans regarder les signes. C'est pourquoi toutes les machines actuelles codent les entiers relatifs par complément à deux, et c'est ce codage que l'on étudie.

Le tableau 4.1 montre le codage sur 4 bits, où l'intervalle signé va de -8 à $+7$: la moitié haute des mots (bit de poids fort 1) porte les négatifs.

Mot	0000	0001	0010	0100	0111
Valeur	0	1	2	4	7
Mot	1000	1010	1100	1110	1111
Valeur	-8	-6	-4	-2	-1

Table 4.1 – Extraits du codage signé sur 4 bits (complément à deux)

Exemple résolu 4.1 : coder puis relire un nombre négatif

Codons -77 sur 8 bits. On écrit d'abord $77 = (0100\,1101)_2$, car $64 + 8 + 4 + 1 = 77$. L'inversion des bits donne 1011 0010, et l'ajout de 1 fournit 1011 0011 : le complément à deux de -77 sur 8 bits est $(1011\,0011)_2$. Pour relire ce mot : son interprétation non signée vaut $128 + 32 + 16 + 2 + 1 = 179$, le bit de poids fort vaut 1, donc le nombre codé vaut $179 - 256 = -77$. Sur 8 bits, les entiers signés occupent ainsi l'intervalle de -128 à $+127$.

Attention

Ne pas confondre le *complément à un* (la simple inversion des bits) et le *complément à deux*, qui ajoute 1 après l'inversion. Le complément à un de 77 sur 8 bits vaut 1011 0010, soit -78 en lecture signée ; le complément à deux vaut 1011 0011, soit -77. Oublier l'ajout final de 1 est l'erreur la plus fréquente du codage : elle décale chaque résultat négatif d'une unité, et donne notamment -0 pour le complément à un de zéro, un code que le complément à deux, lui, ne connaît pas.

Exemple résolu 4.2 : additionner en complément à deux

Calculons $5 + (-3)$ sur 8 bits. On code $5 = (0000\,0101)_2$ et -3 par complément à deux : $3 = (0000\,0011)_2$, inversé en $(1111\,1100)_2$, plus 1 donne $(1111\,1101)_2$. L'addition binaire donne

$$(0000\,0101)_2 + (1111\,1101)_2 = (1\,0000\,0010)_2.$$

Le neuvième bit, sorti du mot, est ignoré par la machine : il reste $(0000\,0010)_2 = 2$, le bon résultat. C'est tout l'intérêt du complément à deux : le même circuit d'addition traite les nombres positifs et négatifs, et la soustraction se réduit à l'addition du complément.

Les nombres à virgule s'écrivent de deux façons. En *virgule fixe*, on réserve un nombre fixé de bits pour la partie fractionnaire : sur 8 bits partagés en 4 bits entiers et 4 bits fractionnaires, 6,5 s'écrit $(0110\,1000)_2$, car 0110 vaut 6 et 1000 vaut $\frac{8}{16} = 0{,}5$. En *virgule flottante*, normalisée par la norme IEEE 754, un mot de 32 bits contient un bit de signe, 8 bits d'exposant et 23 bits de mantisse, et un mot de 64 bits les double : c'est le format utilisé par défaut pour les nombres réels dans tous les langages actuels.

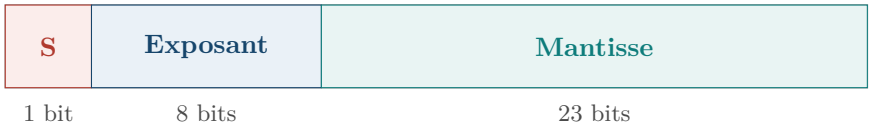


Figure 4.1 – Un mot de virgule flottante simple précision (32 bits, norme IEEE 754)

2 Mémoire d'un ordinateur

Définition 4.2 : octet, adresse

La mémoire vive est une suite d'octets, chacun repéré par un numéro appelé *adresse*, qui commence à 0. Une variable occupe un ou plusieurs octets consécutifs, et son adresse est celle de son premier octet. Si le bus d'adresses comporte n bits, le processeur peut désigner 2^n adresses distinctes : un bus de 16 bits atteint $2^{16} = 65\,536$ octets, un bus de 32 bits

$2^{32} = 4\,294\,967\,296$ octets, soit 4 Go de mémoire adressable.

Les capacités se calculent avec les multiples de la section 3 : chez le fabricant, 1 Ko $= 10^3$ octets, 1 Mo $= 10^6$, 1 Go $= 10^9$; pour la mémoire vive, les puissances de 2 priment, avec $2^{10} = 1024$, $2^{20} = 1\,048\,576$ et $2^{30} = 1\,073\,741\,824$. Une « barrette de 8 Go » contient ainsi $8 \times 2^{30} = 8\,589\,934\,592$ octets. La machine organise en outre ses mémoires en hiérarchie, du plus rapide au plus lent : chaque niveau sacrifie de la vitesse à la capacité, comme le résume le tableau 4.2.

Niveau	Capacité typique	Temps d'accès (ordre)	Volatile ?
Registres du processeur	quelques dizaines d'octets	~ 1 ns	oui
Mémoire cache	Ko à quelques Mo	~ 10 ns	oui
Mémoire vive (RAM)	4 à 32 Go	~ 100 ns	oui
Disque SSD	128 Go à 2 To	$\sim 0{,}1$ ms	non
Disque dur (HDD)	1 à 8 To	~ 10 ms	non

Table 4.2 – La hiérarchie mémoire : la vitesse diminue quand la capacité grandit

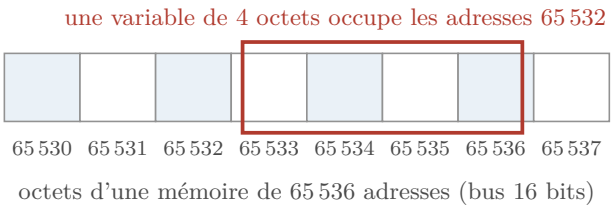


Figure 4.2 – La mémoire vive comme suite d'octets adressés : une variable 32 bits finit au dernier octet

Cette hiérarchie explique un conseil constant des programmeurs : garder au plus près du processeur les données utilisées souvent, et ne descendre vers la RAM ou le disque qu'en cas de besoin. Les « mémoires volatiles » perdent leur contenu dès la coupure de courant ; seuls les disques conservent l'information hors tension.

Exemple résolu 4.3 : compter les adresses et la capacité réelle

Un microcontrôleur de carte à puce possède un bus d'adresses de 16 bits : il adresse $2^{16} = 65\,536$ octets, soit 64 Ko de mémoire, numérotés de 0 à 65 535. Une variable de type entier 32 bits déclarée à l'adresse 65 532 occupe les octets 65 532 à 65 535 : elle finit exactement au dernier octet adressable. Une clé annoncée 32 Go contient en réalité $32 \times 10^9 = 32\,000\,000\,000$ octets ; comptés en puissances de 2, cela fait $32\,000\,000\,000 / 2^{30} \approx 29{,}8$ « Gio », ce qui explique l'écart que l'explorateur de fichiers affiche entre la capacité annoncée et la capacité vue par le système.

Exemple résolu 4.4 : estimer ce que contient une clé

Combien de photographies de 4 Mo peut stocker la clé de 32 Go ? En octets de fabricant, la clé vaut 32×10^9 octets et chaque photo 4×10^6 octets, d'où

$$\frac{32 \times 10^9}{4 \times 10^6} = 8\,000.$$

Huit mille photographies tiennent donc sur la clé, avant l'espace réservé par le système de

fichiers. Le même calcul s'applique à un fichier vidéo de 1,5 Go sur un disque de 500 Go : $500 \times 10^9 / (1{,}5 \times 10^9) \approx 333$ vidéos.

Une confusion d'étudiants mérite d'être désamorcée dès maintenant : « mémoire » et « stockage » ne désignent pas la même chose. La mémoire vive (RAM) garde les programmes et les données *pendant* l'exécution, et perd tout à la coupure de courant ; le stockage (SSD, disque dur) conserve les fichiers *entre* les sessions, mais il est plus lent. Ouvrir un fichier le recopie du stockage vers la mémoire vive ; enregistrer le recopie en sens inverse. Les deux capacités, « 8 Go de RAM » et « 512 Go de stockage », répondent donc à deux besoins distincts et ne se remplacent pas.

3 Limites de précision

La mémoire étant finie, tout codage atteint ses limites quelque part. La première limite est le *dépassement de capacité* (*overflow*) : un résultat qui sort de l'intervalle du type utilisé est tronqué et donne une réponse fausse, sans aucun message d'erreur.

Exemple résolu 4.5 : un dépassement silencieux

Sur 8 bits signés, l'intervalle va de -128 à $+127$. Calculons $100 + 50$: on code $100 = (0110\,0100)_2$ et $50 = (0011\,0010)_2$, et l'addition binaire donne $(1001\,0110)_2$. Relu comme mot signé, ce mot vaut $150 - 256 = -106$: la machine affiche $100 + 50 = -106$, sans avertissement. Le bit de poids fort, qui marque le signe, a été envahi par la retenue du calcul. Tout programmeur vérifie donc l'intervalle du type choisi avant d'y accumuler des valeurs.

La seconde limite concerne la virgule flottante : les nombres réels sont représentés en nombre fini de bits, donc de façon approchée. Le nombre 0,1 n'a pas d'écriture binaire finie, de même que $\frac{1}{3}$ n'a pas d'écriture décimale finie, et la machine le stocke par une valeur très proche mais inexacte.

Exemple résolu 4.6 : quand la machine se trompe de 0,0000000000000001

Dans un interpréteur Python, l'expression $0{,}1 + 0{,}2$ renvoie exactement :

```
>>> 0.1 + 0.2
0.30000000000000004
```

L'erreur est minuscule, mais elle suffit à faire échouer un test : l'égalité `0.1 + 0.2 == 0.3` renvoie `False`, car 0,3 stocké n'est pas exactement le résultat stocké de $0{,}1+0{,}2$. En pratique, on compare des flottants avec une tolérance (déclarer le résultat correct si $|x - 0{,}3| < 10^{-9}$), et on ne code jamais des montants financiers en flottant.

Remarque

[l'accumulation des petites erreurs] Une erreur minuscule devient visible quand elle s'accumule. Additionner 1000 fois la valeur 0,1 devrait donner 100 ; un interpréteur Python renvoie 99,9999999999986, avec un déficit d'environ $1{,}4 \times 10^{-12}$. Sur un calcul isolé, l'écart est négligeable ; sur une boucle de paie, de capitalisation ou de trajectoire, il peut devenir gênant. Les règles du métier suivent : comparer avec tolérance, préférer les entiers dès que la donnée le permet, et réserver les flottants aux quantités réellement continues.

Attention

Deux réflexes protègent de ces limites. D'une part, toute accumulation doit être confrontée à l'intervalle du type : mille ajouts de 3 millions sur un entier 16 bits signé (max 32 767) débordent dès le premier. D'autre part, l'égalité de deux flottants ne se teste jamais directement : deux calculs mathématiquement identiques peuvent différer au dernier bit, et la comparaison rigoureuse passe par une tolérance absolue ou relative.

Remarque

Comment détecter un dépassement sans recalculer tout le tableau ? Pour l'addition signée en complément à deux, on observe les deux retenues sortantes : celle qui quitte le bit de poids fort et celle qui entre dans la colonne du signe. Si ces deux retenues sont identiques (toutes deux 0 ou toutes deux 1), le résultat est correct ; si elles diffèrent, il y a dépassement. Dans l'exemple $100 + 50$, la retenue entrant dans le signe vaut 1 et celle sortant du mot vaut 0 : le dépassement est détecté, et un processeur le signale en levant son drapeau d'erreur, le fameux *overflow flag* que les tableaux du chapitre 3 laissaient pressentir.

4 Choix de la bonne représentation

Choisir une représentation, c'est répondre à trois questions : la donnée est-elle un entier ou un réel ? quel intervalle de valeurs doit couvrir le type ? quelle place chaque enregistrement consommera-t-il, multipliée par le nombre d'enregistrements ?

Méthode : choisir un type : trois questions

1. la donnée est-elle un comptage (entier) ou une mesure (réel) ? un âge, une population, un matricule sont des entiers ; une moyenne, un prix, une coordonnée sont des réels, ou des entiers de plus petite unité (centimes, centièmes de point) ;
2. quel intervalle doit couvrir le type ? reprendre le tableau 4.3 et garder une marge : un compteur d'octets d'un disque de 2 To dépasse l'entier 32 bits signé ($\approx$ 2,1 milliards) dès le premier téraoctet ;
3. quelle place par enregistrement, multipliée par combien d'enregistrements ? la différence entre 2 et 8 octets se paie sur des fichiers de dizaines de milliers de lignes.

Type	Intervalle couvert	Usage typique
Entier 8 bits signé	-128 à 127	compteur court, échantillon audio
Entier 16 bits signé	$-32\,768$ à 32 767	année, pixel d'une composante RVB
Entier 32 bits signé	$\approx \pm 2{,}1$ milliards	identifiant, population d'un pays
Entier 64 bits signé	$\approx \pm 9{,}2$ milliards de milliards	horodatage, compteur universel
Flottant 32 bits	$\approx \pm 3{,}4 \times 10^{38}$, 7 chiffres fiables	graphique, capteur
Flottant 64 bits	$\approx \pm 1{,}8 \times 10^{308}$, 15 chiffres fiables	calcul scientifique par défaut

Table 4.3 – Intervalles et usages des représentations usuelles

Exemple résolu 4.7 : choisir un type pour une donnée réelle

Un système de gestion des étudiants de CPTEC doit stocker un matricule, un âge et une moyenne. Le matricule, entier positif de moins de 9 chiffres, tient sur un entier 32 bits non signé ; l'âge tient largement sur un octet ; la moyenne, réel compris entre 0 et 20 avec deux décimales, se code soit en flottant 64 bits avec la tolérance de comparaison vue plus haut, soit, mieux pour des notes réglementées, en entier de centièmes : une moyenne de 14,75 devient l'entier 1475, exact et sans erreur d'arrondi. Dans un fichier de 50 000 étudiants, passer la moyenne du flottant 64 bits (8 octets) à l'entier de centièmes sur 16 bits (2 octets) économise $50\,000 \times 6 = 300\,000$ octets : le choix du type se paie en octets.

Exemple résolu 4.8 : une image occupe aussi des octets

L'écran de la salle de travaux pratiques affiche 1920×1080 pixels en couleurs RVB (24 bits par pixel, soit 3 octets). La mémoire d'affichage, le *framebuffer*, doit donc contenir

$$1920 \times 1080 \times 3 = 6\,220\,800 \text{ octets} \approx 6{,}2 \text{ Mo}.$$

Chaque image affichée occupe ainsi 6,2 Mo de mémoire, et un vidéo-projecteur qui rafraîchit l'écran 60 fois par seconde lit chaque seconde $60 \times 6{,}2 \approx 372$ Mo : le simple affichage d'un bureau vide mobilise déjà une part du débit mémoire. Le chapitre 6 reprend ce calcul pixel par pixel.

Ce qu'il faut retenir

Sur n bits : 0 à $2^n - 1$ en non signé, -2^{n-1} à $2^{n-1} - 1$ en signé. Le complément à deux s'obtient en inversant les bits puis en ajoutant 1 ; sa lecture retranche 2^n lorsque le bit de poids fort vaut 1 ; l'addition signée utilise le même circuit, le bit sortant étant ignoré. La mémoire est une suite d'octets adressés de 0 à $2^n - 1$ pour un bus de n bits ; 1 Ko $= 10^3$ octets chez les fabricants contre $2^{10} = 1024$ pour le système. Deux pièges : le dépassement de capacité, qui donne $100 + 50 = -106$ sur 8 bits signés sans prévenir, et la précision finie des flottants, où $0{,}1 + 0{,}2 \neq 0{,}3$; on compare donc avec une tolérance et on code les montants en entiers de centimes. Le type se choisit selon l'intervalle à couvrir et la place multipliée par le nombre d'enregistrements.

Chapitre 5

Mesures des angles

En quelques lignes

Les angles mesurent les rotations, et l'informatique tourne autour des rotations : une image qu'on redresse, un personnage de jeu qui pivote, une antenne qu'on oriente vers une source de signal, un bras de robot qui s'articule. Ce chapitre fixe les deux unités de mesure (le degré et le radian) et leur conversion, puis reprend les triangles rectangles avec le théorème de Pythagore et les rapports trigonométriques, qui calculent longueurs et distances inaccessibles. La dernière section introduit les vecteurs du plan, objets qui portent à la fois une direction, un sens et une longueur, et préparent le chapitre 6 sur les images numériques.

1 Mesure des angles

Définition 5.1 : angle, degré, radian

Un *angle* est la figure formée par deux demi-droites de même origine, appelée sommet. Il se mesure en *degrés* : un tour complet vaut 360°, un angle droit 90° et un angle plat 180°. Le *radian* est la seconde unité, celle des calculs : un angle de 1 radian correspond à un arc de cercle dont la longueur égale le rayon, et un tour complet vaut 2π radians.

Propriété 5.1 : conversion degrés-radians

Puisque $180° = \pi$ rad, la conversion se fait au prorata :

$$\alpha \text{ rad} = \alpha \times \frac{180}{\pi}\,° \qquad \text{et} \qquad \beta\,° = \beta \times \frac{\pi}{180} \text{ rad}.$$

Degrés	0°	30°	45°	60°	90°	180°	360°
Radians	0	$\frac{\pi}{6}$	$\frac{\pi}{4}$	$\frac{\pi}{3}$	$\frac{\pi}{2}$	π	2π

Table 5.1 – Angles remarquables en degrés et en radians

Exemple résolu 5.1 : convertir dans les deux sens

Convertions 150° en radians, puis $\frac{5\pi}{6}$ et 72° dans l'autre sens. Pour 150° : $150 \times \frac{\pi}{180} = \frac{150\pi}{180} = \frac{5\pi}{6}$ rad, en simplifiant par 30. Réciproquement, $\frac{5\pi}{6}$ rad vaut $\frac{5\pi}{6} \times \frac{180}{\pi} = \frac{5 \times 180}{6} = 150°$. Pour 72° : $72 \times \frac{\pi}{180} = \frac{2\pi}{5}$ rad. On remarque que les deux sens de conversion utilisent les mêmes fractions : $\frac{\pi}{180}$ pour descendre en radians, $\frac{180}{\pi}$ pour remonter en degrés.

Un cadran d'horloge donne à la notion d'angle un terrain d'exercice quotidien : chaque heure vaut $360°/12 = 30°$ et chaque minute $360°/60 = 6°$. À 3 h 30, la petite aiguille se trouve à

mi-chemin entre 3 et 4, soit $3{,}5 \times 30° = 105°$ depuis midi, et la grande à $30 \times 6° = 180°$; l'angle entre les deux aiguilles vaut donc $180° - 105° = 75°$.

Exemple résolu 5.2 : faire tourner un pixel de 90°

Faire tourner un point du plan de 90° dans le sens direct (sens inverse des aiguilles d'une montre) transforme le point de coordonnées (x, y) en $(-y, x)$. Le pixel situé en $(4, 1)$ se retrouve ainsi en $(-1, 4)$ après un quart de tour, et le point $(-1, 4)$ revient à $(4, 1)$ après trois quarts de tour supplémentaires, soit $3 \times 90° = 270° = \dfrac{3\pi}{2}$ rad. Les logiciels de dessin et les moteurs de jeux reçoivent d'ailleurs leurs angles en radians par défaut : un quart de tour s'y demande avec l'angle $\pi/2$, et la conversion du tableau 5.1 devient immédiatement utile.

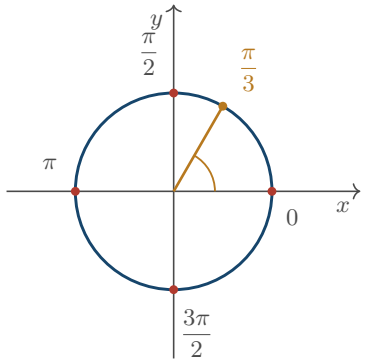


Figure 5.1 – Le cercle trigonométrique de rayon 1 : les angles remarquables se lisent en radians

2 Les triangles

Propriété 5.2 : somme des angles et théorème de Pythagore

Dans tout triangle, la somme des trois angles vaut 180°. Dans un triangle *rectangle* en C, dont les côtés adjacents à l'angle droit ont pour longueurs a et b et dont l'hypoténuse a pour longueur c :

$$c^2 = a^2 + b^2,$$

et l'hypoténuse est le côté le plus long, opposé à l'angle droit.

Le théorème de Pythagore s'applique dès qu'une diagonale traverse un rectangle : écran, fenêtre, terrain, feuille de papier. C'est aussi lui qui donne la distance entre deux points du plan, vue à la section 3. Le triangle rectangle de côtés 3, 4 et 5 (car $3^2 + 4^2 = 9 + 16 = 25 = 5^2$) est le plus célèbre des triangles de Pythagore : les maçons l'utilisent depuis l'Antiquité pour tracer un angle droit avec une corde à treize nœuds, et une échelle de 5 m posée à 3 m d'un mur atteint exactement 4 m de hauteur.

Type de triangle	Caractéristique	Exemple d'angles
Équilatéral	trois côtés égaux	60° ; 60° ; 60°
Isocèle	deux côtés égaux	40° ; 70° ; 70°
Quelconque (scalène)	trois côtés inégaux	50° ; 60° ; 70°
Rectangle	un angle droit de 90°	90° ; 30° ; 60°

Table 5.2 – Types de triangles et exemple d'angles (somme toujours 180°)

Pour les triangles quelconques, la *loi des cosinus* généralise Pythagore : dans un triangle dont deux côtés ont pour longueurs b et c et dont l'angle entre eux vaut α, le troisième côté vaut a avec

$$a^2 = b^2 + c^2 - 2\,b\,c\,\cos\alpha.$$

Lorsque $\alpha = 90°$, le terme correctif disparaît et l'on retrouve $a^2 = b^2 + c^2$: Pythagore est le cas particulier d'un angle droit. À titre d'illustration, avec $b = 7$, $c = 8$ et $\alpha = 60°$: $a^2 = 49 + 64 - 2 \times 7 \times 8 \times 0{,}5 = 57$, d'où $a = \sqrt{57} \approx 7{,}55$.

Propriété 5.3 : rapports trigonométriques

Dans un triangle rectangle, pour un angle aigu α :

$$\cos\alpha = \frac{\text{côté adjacent}}{\text{hypoténuse}}, \qquad \sin\alpha = \frac{\text{côté opposé}}{\text{hypoténuse}}, \qquad \tan\alpha = \frac{\text{côté opposé}}{\text{côté adjacent}}.$$

α	0°	30°	45°	60°	90°
$\sin\alpha$	0	$\frac{1}{2}$	$\frac{\sqrt{2}}{2}$	$\frac{\sqrt{3}}{2}$	1
$\cos\alpha$	1	$\frac{\sqrt{3}}{2}$	$\frac{\sqrt{2}}{2}$	$\frac{1}{2}$	0
$\tan\alpha$	0	$\frac{1}{\sqrt{3}}$	1	$\sqrt{3}$	indéfini

Table 5.3 – Valeurs exactes des rapports trigonométriques des angles remarquables

Méthode : résoudre un triangle rectangle

1. repérer l'angle droit et l'angle connu (ou cherché) α ;
2. nommer les trois côtés du point de vue de α : opposé, adjacent, hypoténuse ;
3. choisir le rapport qui relie les deux côtés connus : cos pour adjacent-hypoténuse, sin pour opposé-hypoténuse, tan pour opposé-adjacent ; Pythagore lorsque les trois côtés sont concernés ;
4. isoler l'inconnue, calculer, puis contrôler l'ordre de grandeur : l'hypoténuse doit rester le côté le plus long.

Exemple résolu 5.3 : la diagonale d'un écran par Pythagore

Un écran de 24 pouces au format 16 :9 a une diagonale de 24 pouces ; quelles sont sa largeur et sa hauteur ? Écrivons la largeur $w = 16k$ et la hauteur $h = 9k$, avec k inconnu. Pythagore

donne :

$$(16k)^2 + (9k)^2 = 24^2 \quad \Longrightarrow \quad k^2\,(256+81) = 576 \quad \Longrightarrow \quad k = \frac{24}{\sqrt{337}} \approx 1{,}307.$$

D'où $w \approx 16 \times 1{,}307 \approx 20{,}9$ pouces et $h \approx 9 \times 1{,}307 \approx 11{,}8$ pouces, soit environ 53 cm sur 30 cm (1 pouce = 2,54 cm).

Exemple résolu 5.4 : une hauteur inaccessible par la trigonométrie

Un technicien situe un point d'observation à 15 m du pied d'une antenne, au sol horizontal ; l'angle entre le sol et la ligne de visée du sommet vaut 35°. Quelle est la hauteur h de l'antenne ? Le triangle est rectangle, l'angle connu est adjacent au côté de 15 m et opposé au côté h :

$$\tan 35^\circ = \frac{h}{15} \quad \Longrightarrow \quad h = 15 \times \tan 35^\circ \approx 15 \times 0{,}700 \approx 10{,}5 \text{ m}.$$

L'antenne culmine à environ 10,5 mètres. La même lecture du tableau 5.3 (adjacent, opposé, hypoténuse) résout tous les triangles où une longueur reste inaccessible à la mesure directe.

Remarque

[pourquoi le radian] Le radian n'est pas une unité de plus par caprice : c'est l'unité naturelle du cercle. Un angle de α radians découpé sur un cercle de rayon r correspond à un arc de longueur $s = r \times \alpha$: la longueur se lit directement, sans coefficient de conversion. Un angle de 1 radian sur un cercle de 1 m y découpe un arc d'exactement 1 m ; le même angle sur un cercle de 3 m y découpe 3 m. Cette simplicité fait du radian l'unité des logiciels de graphisme, des moteurs de jeu et de toute la physique des rotations.

3 Les vecteurs

Définition 5.2 : vecteur du plan

Un *vecteur* du plan est un déplacement caractérisé par trois données : une direction, un sens et une longueur (*norme*). Il se représente par une flèche et se note $\vec{u}$; ses *coordonnées* $\begin{pmatrix} x \\ y \end{pmatrix}$ indiquent le déplacement horizontal et le déplacement vertical qu'il effectue. La *norme* de $\vec{u} = \begin{pmatrix} x \\ y \end{pmatrix}$ vaut

$$\|\vec{u}\| = \sqrt{x^2 + y^2},$$

conséquence directe du théorème de Pythagore.

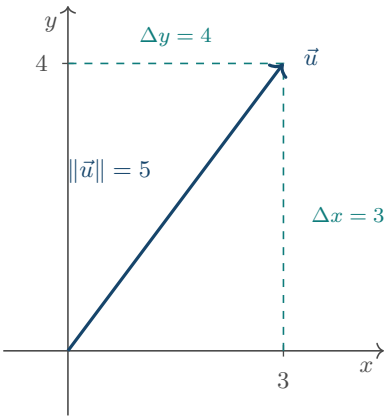


Figure 5.2 – Le vecteur $\vec{u} = \begin{pmatrix} 3 \\ 4 \end{pmatrix}$: ses coordonnées et sa norme

Deux vecteurs s'additionnent coordonnée par coordonnée, et se multiplient par un nombre qui agrandit ou rétrécit leur longueur : $\vec{u} + \vec{v} = \begin{pmatrix} x_u + x_v \\ y_u + y_v \end{pmatrix}$ et $\lambda\vec{u} = \begin{pmatrix} \lambda x \\ \lambda y \end{pmatrix}$. Deux vecteurs sont *colinéaires* lorsque l'un est multiple de l'autre : ils ont la même direction, au sens près ou à sens contraire. En informatique, un vecteur désigne aussi bien un déplacement sur l'écran qu'une liste de données ; le mot désigne ici l'objet géométrique, le chapitre 6 reprendra la lecture informatique. Sur un écran de 1920×1080 pixels, le déplacement d'un curseur de dix pixels vers la droite et quatre vers le bas s'écrit simplement le vecteur $\begin{pmatrix} 10 \\ 4 \end{pmatrix}$, de norme $\sqrt{100 + 16} = \sqrt{116} \approx 10{,}8$ pixels.

En informatique, la norme d'un vecteur sert dès qu'une distance se calcule : distance du curseur à un objet cliqué, longueur d'un déplacement par image dans un jeu, taille d'un pas d'algorithme. La direction du vecteur sert à orienter : un personnage se tourne vers la direction de son vecteur de vitesse, un projectile part dans la direction du vecteur visé. Un même objet mathématique, trois usages : déplacer, mesurer, orienter.

Propriété 5.4 : produit scalaire et angle entre deux vecteurs

Le *produit scalaire* de deux vecteurs $\vec{u} = \begin{pmatrix} x_u \\ y_u \end{pmatrix}$ et $\vec{v} = \begin{pmatrix} x_v \\ y_v \end{pmatrix}$ vaut $\vec{u} \cdot \vec{v} = x_u\, x_v + y_u\, y_v$, et il relie la norme et l'angle θ entre les deux vecteurs :

$$\vec{u} \cdot \vec{v} = \|\vec{u}\| \times \|\vec{v}\| \times \cos\theta.$$

Un produit scalaire nul signale deux vecteurs perpendiculaires, puisque $\cos 90° = 0$.

À titre d'illustration, les vecteurs $\begin{pmatrix} 3 \\ 4 \end{pmatrix}$ et $\begin{pmatrix} 4 \\ 3 \end{pmatrix}$ ont pour produit scalaire $3 \times 4 + 4 \times 3 = 24$, et comme chacun a pour norme 5, on lit $\cos\theta = 24/25 = 0{,}96$, d'où un angle d'environ 16,3° entre eux : deux directions proches, comme le confirme un dessin rapide.

Exemple résolu 5.5 : opérer sur des vecteurs

Soient $\vec{u} = \begin{pmatrix} 3 \\ 4 \end{pmatrix}$ et $\vec{v} = \begin{pmatrix} -1 \\ 2 \end{pmatrix}$. La somme vaut $\vec{u} + \vec{v} = \begin{pmatrix} 3 + (-1) \\ 4 + 2 \end{pmatrix} = \begin{pmatrix} 2 \\ 6 \end{pmatrix}$, la norme de $\vec{u}$ vaut $\|\vec{u}\| = \sqrt{3^2 + 4^2} = \sqrt{25} = 5$, et le double $2\vec{u} = \begin{pmatrix} 6 \\ 8 \end{pmatrix}$ est colinéaire à $\vec{u}$, de norme 10 : le même déplacement, deux fois plus long.

Exemple résolu 5.6 : distance entre deux points du plan image

Quelle est la distance entre le pixel A de coordonnées $(2, 3)$ et le pixel B de coordonnées $(7, 15)$ d'une image ? Le déplacement de A vers B est le vecteur $\overrightarrow{AB} = \begin{pmatrix} 7 - 2 \\ 15 - 3 \end{pmatrix} = \begin{pmatrix} 5 \\ 12 \end{pmatrix}$, dont la norme vaut

$$\|\overrightarrow{AB}\| = \sqrt{5^2 + 12^2} = \sqrt{25 + 144} = \sqrt{169} = 13.$$

La distance entre les deux points vaut 13 pixels. Ce calcul (différence de coordonnées, puis Pythagore) est celui que toute bibliothèque graphique exécute pour mesurer, aligner ou animer.

Attention

Sur l'écran, deux conventions de coordonnées cohabitent et ne doivent jamais être mélangées. Les mathématiques du plan placent l'origine en bas à gauche, l'axe des y vers le haut ; les bibliothèques d'images, elles, comptent les lignes depuis le haut de l'image, l'origine en haut à gauche, l'axe vertical vers le bas. Le pixel « ligne 3, colonne 7 » d'une photo n'est donc pas le point de coordonnées $(7, 3)$ du repère du cours, mais son symétrique vertical. Avant tout calcul d'image, s'assurer de la convention du logiciel utilisé.

Ce qu'il faut retenir

Un tour complet vaut 360° ou 2π rad, et la conversion se fait par $\pi = 180°$: multiplier par $\frac{\pi}{180}$ pour passer en radians, par $\frac{180}{\pi}$ pour revenir en degrés (tableau 5.1). Dans un triangle rectangle : somme des angles 180°, Pythagore $c^2 = a^2 + b^2$, et les rapports cos, sin, tan relient les côtés (tableau 5.3) : la diagonale d'un écran 16 :9 et la hauteur d'une antenne se calculent ainsi. Un vecteur $\begin{pmatrix} x \\ y \end{pmatrix}$ porte direction, sens et norme $\sqrt{x^2 + y^2}$; la somme se fait coordonnée par coordonnée, la colinéarité signe la même direction, et la distance entre deux points est la norme du vecteur qui les relie. Le chapitre 6 transpose ces vecteurs aux images : chaque pixel y devient une adresse, chaque couleur une donnée.

Chapitre 6

Les vecteurs

En quelques lignes

Le mot « vecteur » désigne aussi, en informatique, une liste ordonnée de nombres ; et une image numérique n'est rien d'autre qu'une grande matrice de nombres. Ce chapitre commence par les vecteurs et les matrices en tant qu'objets de calcul (dimensions, somme, produit), puis les applique aux images : différence entre image vectorielle et image matricielle, taille et résolution, codage des couleurs en RVB, et calcul du poids d'un fichier. Le chapitre 5 fournit la norme des vecteurs et le théorème de Pythagore utilisés pour les dimensions.

1 Définition de vecteurs et de matrices

Définition 6.1 : vecteur, matrice

Un *vecteur* de dimension n est une liste ordonnée de n nombres, notée en colonne ou en ligne. Une *matrice* de taille $m \times n$ est un tableau de m lignes et n colonnes de nombres ; son élément de ligne i et colonne j se note a_{ij}. Une image numérique de 1080 lignes et 1920 colonnes est une matrice 1080×1920.

En mémoire, les éléments d'une matrice se rangent ligne après ligne : l'image commence par sa première ligne de pixels, puis la seconde, et ainsi de suite jusqu'à la dernière. Ce rangement détermine la vitesse des traitements : parcourir une image ligne par ligne suit l'ordre de la mémoire et reste rapide, tandis que la parcourir colonne par colonne saute d'un bout à l'autre de la mémoire à chaque pas.

La somme de deux vecteurs de même dimension, et celle de deux matrices de même taille, se font élément par élément. Le produit d'une matrice A de taille $m \times n$ par un vecteur $\vec{v}$ de dimension n produit un vecteur de dimension m dont chaque coordonnée est la somme des produits d'une ligne de A par les coordonnées de $\vec{v}$:

$$\begin{pmatrix} a & b \\ c & d \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} a\,x + b\,y \\ c\,x + d\,y \end{pmatrix}.$$

Deux matrices se multiplient selon la règle « ligne de la première, colonne de la seconde » : le produit $A \times B$ d'une matrice $m \times n$ par une matrice $n \times p$ donne une matrice $m \times p$ dont l'élément de ligne i et colonne j est la somme des produits de la ligne i de A par la colonne j de B. Sur des matrices 2×2 :

$$\begin{pmatrix} a & b \\ c & d \end{pmatrix} \begin{pmatrix} e & f \\ g & h \end{pmatrix} = \begin{pmatrix} a\,e + b\,g & a\,f + b\,h \\ c\,e + d\,g & c\,f + d\,h \end{pmatrix}.$$

Composer deux transformations revient à multiplier leurs matrices : si B fait tourner et A fait changer d'échelle, la matrice produit $A \times B$ fait les deux d'un coup.

Exemple résolu 6.1 : opérations élémentaires

Soient $\vec{u} = \begin{pmatrix} 2 \\ 5 \end{pmatrix}$, $\vec{v} = \begin{pmatrix} -3 \\ 1 \end{pmatrix}$ et $A = \begin{pmatrix} 2 & 0 \\ 0 & 3 \end{pmatrix}$. La somme vaut $\vec{u} + \vec{v} = \begin{pmatrix} 2 + (-3) \\ 5 + 1 \end{pmatrix} = \begin{pmatrix} -1 \\ 6 \end{pmatrix}$. Le produit matrice-vecteur vaut

$$A\,\vec{u} = \begin{pmatrix} 2 \times 2 + 0 \times 5 \\ 0 \times 2 + 3 \times 5 \end{pmatrix} = \begin{pmatrix} 4 \\ 15 \end{pmatrix}.$$

Chaque coordonnée a été multipliée par un facteur différent : la matrice A a étiré le vecteur de 2 en horizontal et de 3 en vertical.

Exemple résolu 6.2 : matrices géométriques : échelle et rotation

Deux matrices carrées décrivent des transformations de base du plan. La matrice d'*échelle* $S = \begin{pmatrix} k & 0 \\ 0 & k \end{pmatrix}$ multiplie toute norme par k : avec $k = 2$, le pixel $\begin{pmatrix} 40 \\ 25 \end{pmatrix}$ devient $\begin{pmatrix} 80 \\ 50 \end{pmatrix}$, l'image double de taille. La matrice de *rotation* de 90° dans le sens direct $R = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}$ transforme $\begin{pmatrix} x \\ y \end{pmatrix}$ en $\begin{pmatrix} -y \\ x \end{pmatrix}$: le pixel $\begin{pmatrix} 4 \\ 1 \end{pmatrix}$ devient $\begin{pmatrix} -1 \\ 4 \end{pmatrix}$, exactement la rotation étudiée au chapitre 5. Tout logiciel de retouche enchaîne ces matrices à chaque clic sur « redimensionner » ou « pivoter ». Leur produit compose les deux opérations :

$$S \times R = \begin{pmatrix} 2 & 0 \\ 0 & 3 \end{pmatrix} \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix} = \begin{pmatrix} 0 & -2 \\ 3 & 0 \end{pmatrix},$$

et l'on vérifie que ce produit appliqué au pixel $\begin{pmatrix} 4 \\ 1 \end{pmatrix}$ donne $\begin{pmatrix} -2 \\ 12 \end{pmatrix}$, le même résultat que la rotation suivie de l'échelle : le pixel a tourné d'un quart de tour puis a été étiré. Une seule multiplication matricielle remplace deux transformations successives, et c'est ainsi que les logiciels traitent des millions de pixels à chaque action.

2 Image vectorielle et image matricielle

Définition 6.2 : image matricielle, image vectorielle

Une *image matricielle* (ou bitmap) décrit une grille de pixels, chaque case portant sa couleur : l'image est un tableau géant, comme la matrice de la section 1. Une *image vectorielle* décrit des formes par des équations (segments, cercles, courbes de Bézier, aires remplies) dont le logiciel recalcule le rendu à chaque affichage.

Les deux familles ne se comportent pas de la même façon à l'agrandissement. L'image matricielle, zoomée, montre ses pixels : chaque case devient un carré visible, et la photo paraît crénelée ; l'image vectorielle, elle, se redessine sans aucune perte à n'importe quel facteur d'agrandissement, car ses équations ne dépendent pas de la taille d'affichage. En contrepartie, une image matricielle représente sans effort une photographie aux millions de nuances, alors qu'un dessin vectoriel

complexe demande des milliers d'équations.

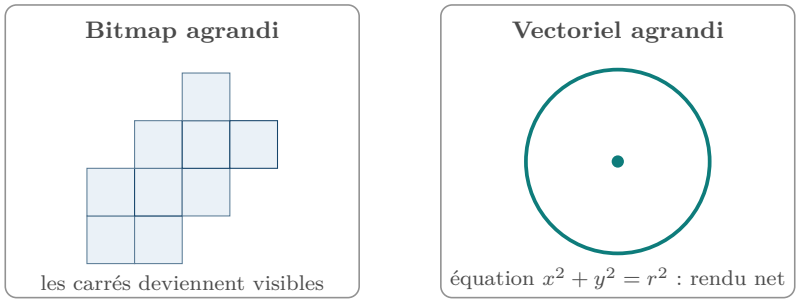


Figure 6.1 – À l'agrandissement, le bitmap montre ses pixels, le vectoriel se redessine

Critère	Image matricielle	Image vectorielle
Contenu	grille de pixels colorés	équations de formes
Formats	JPG, PNG, GIF	SVG, AI, PDF vectoriel
Agrandissement	pixels visibles, perte de qualité	rendu net à toute échelle
Usages	photographies, captures d'écran	logos, plans, typographies
Poids	croît avec le nombre de pixels	croît avec la complexité du dessin

Table 6.1 – Image matricielle et image vectorielle face à face

Lue comme objet mathématique, une image matricielle est une fonction de deux indices : $f(i, j)$ donne la couleur du pixel de ligne i et colonne j, avec $0 \leqslant i < 1080$ et $0 \leqslant j < 1920$ pour un écran Full HD. Les traitements d'image (luminosité, contraste, filtre de netteté) réécrivent ces valeurs par des formules appliquées à chaque pixel ou à son voisinage ; la matrice de la section 1 n'est pas une métaphore, c'est la structure de données réelle du fichier en mémoire.

Exemple résolu 6.3 : choisir la bonne famille

Un studio de Lomé prépare deux documents : la couverture d'un rapport, qui mêle photographie de terrain et bandeau de couleur unie, et le logo de l'agence, qui doit figurer en haut d'une lettre aussi bien que sur une banderole de 4 mètres. La photographie passe en image matricielle (JPG), seule capable de restituer les nuances du réel ; le logo passe en vectoriel (SVG), car il sera redessiné à toutes les tailles sans crénelage. Le bon choix dépend donc de l'usage final, non du logiciel disponible.

3 Taille et dimension d'une image numérique

Définition 6.3 : pixel, définition, résolution

Un *pixel* (picture element) est la plus petite unité d'une image matricielle. La *définition* d'une image est son nombre total de pixels, produit du nombre de colonnes par le nombre de lignes : $4000 \times 3000 = 12\,000\,000$ pixels, soit 12 mégapixels (Mpx). La *résolution* d'affichage ou d'impression compte les pixels par pouce (dpi, *dots per inch*) : elle décide de la taille physique du rendu, puisque $L_{\text{cm}} = L_{\text{px}} \times 2{,}54/\text{dpi}$.

La même définition produit des tailles physiques différentes selon la résolution : l'image de 12 Mpx imprimée à 300 dpi donne $\dfrac{4000 \times 2{,}54}{300} \approx 33{,}9$ cm de large, tandis qu'affichée à 72 dpi

elle occupe plus de 1,4 mètre de large sur l'écran. Enfin le *rapport d'affichage* (ou format) est le quotient largeur sur hauteur : un écran 16 :9 a un rapport de $\frac{1920}{1080} \approx 1{,}78$, et une image 4 :3, comme les anciens téléviseurs, un rapport de 1,33 ; afficher une image dans un cadre d'un autre rapport exige de rogner ou d'ajouter des bandes.

Appellation	Dimensions (px)	Définition	Rapport
HD	1280×720	0,92 Mpx	16 :9
Full HD	1920×1080	2,07 Mpx	16 :9
4K UHD	3840×2160	8,29 Mpx	16 :9
Photo smartphone	4000×3000	12 Mpx	4 :3

Table 6.2 – Définitions usuelles : la qualité se mesure en pixels

Exemple résolu 6.4 : définition et rapport d'un écran

Un écran affiche 1920×1080 pixels. Sa définition vaut $1920 \times 1080 = 2\,073\,600$ pixels, soit environ 2,07 Mpx. Son rapport vaut $\frac{1920}{1080} = \frac{16}{9}$, en simplifiant par 120 : c'est un écran 16 :9. Une photo de 4000×3000 a pour sa part un rapport de $\frac{4}{3}$: affichée plein écran sur le 16 :9, elle laisse deux bandes verticales, ou bien elle est recadrée de $\frac{4}{3} \to \frac{16}{9}$ au prix d'une perte de hauteur.

Attention

Ne pas confondre la *définition*, qui compte les pixels de l'image elle-même, et la *résolution*, qui décide de la taille de leur rendu sur un support : la première appartient au fichier, la seconde au périphérique. Une même image de 12 Mpx est « grande » sur un écran de téléphone à 400 pixels par pouce et « immense » sur un vidéo-projecteur à 80 pixels par pouce. Lorsqu'un logiciel propose de changer la résolution sans changer les pixels, il ne touche qu'à la taille d'impression prévue, jamais au contenu de l'image.

Exemple résolu 6.5 : préparer une impression à 300 dpi

On veut imprimer la photo de 4000×3000 pixels à 300 dpi, la résolution d'impression de qualité. La largeur imprimée vaut $\frac{4000 \times 2{,}54}{300} \approx 33{,}9$ cm et la hauteur $\frac{3000 \times 2{,}54}{300} \approx 25{,}4$ cm : la photo déborde d'une page A4 en paysage (29,7 cm sur 21 cm), mais tient en pleine page A3, dont le format paysage offre 42 cm sur 29,7 cm. Si l'on tient la même photo à 150 dpi, la taille double : 67,7 cm sur 50,8 cm, utile pour une affiche vue de loin, où l'œil ne distingue plus les pixels.

4 Codage des couleurs

Définition 6.4 : modèle RVB

Le modèle *RVB* (rouge, vert, bleu) décrit toute couleur comme l'addition de trois composantes lumineuses, chacune codée sur 8 bits, donc entre 0 et 255. Une couleur pèse 24 bits par pixel : $(255, 0, 0)$ est le rouge pur, $(0, 0, 0)$ le noir, $(255, 255, 255)$ le blanc. En notation hexadécimale, très utilisée dans les pages web, chaque canal s'écrit sur deux chiffres hexadécimaux du chapitre 3 : le rouge pur devient `#FF0000`.

Le choix du rouge, du vert et du bleu n'a rien d'arbitraire : la rétine humaine dispose de trois familles de récepteurs sensibles à ces trois plages de lumière, et tout ce que l'œil perçoit se recompose par mélange de ces trois sensations. Un écran qui allume trois luminophores aux couleurs bien choisies trompe l'œil avec la même recette. La synthèse additive des écrans, qui ajoute de la lumière du noir vers le blanc, s'oppose à la synthèse soustractive des imprimantes, qui retranche de la lumière d'un support blanc et mélange cyan, magenta et jaune.

Propriété 6.1 : nombre de couleurs

Sur 24 bits par pixel, le nombre de couleurs distinctes vaut $2^8 \times 2^8 \times 2^8 = 2^{24} = 16\,777\,216$: c'est le « 16 millions de couleurs » des fiches techniques. Une image en *niveaux de gris* n'utilise qu'un canal de 8 bits : 256 nuances, du noir (0) au blanc (255), suffisant pour un document scanné en texte.

Couleur	RVB (R, V, B)	Hexadécimal	Composition lumineuse
Noir	$(0, 0, 0)$	`#000000`	aucune composante
Blanc	$(255, 255, 255)$	`#FFFFFF`	les trois à fond
Rouge pur	$(255, 0, 0)$	`#FF0000`	rouge seul
Jaune	$(255, 255, 0)$	`#FFFF00`	rouge + vert
Cyan	$(0, 255, 255)$	`#00FFFF`	vert + bleu
Magenta	$(255, 0, 255)$	`#FF00FF`	rouge + bleu

Table 6.3 – Couleurs usuelles en RVB : la synthèse additive des écrans

Exemple résolu 6.6 : d'une couleur RVB au code hexadécimal

Convertissons la couleur $(164, 78, 20)$ en hexadécimal. Le canal rouge : $164 = 10 \times 16 + 4$, donc `A4`. Le canal vert : $78 = 4 \times 16 + 14$, donc `4E`, car 14 s'écrit E. Le canal bleu : $20 = 1 \times 16 + 4$, donc `14`. La couleur s'écrit `#A44E14` : un orange-brun sombre, typique d'une palette de terre. Dans l'autre sens, le code `#1A2B3C` se décode en $26, 43, 60$: un bleu nuit.

Exemple résolu 6.7 : compter les couleurs et les nuances

Combien de nuances offre une image en niveaux de gris 8 bits ? Un seul canal : $2^8 = 256$ nuances. Combien de couleurs sur un écran 30 bits (10 bits par canal) ? $2^{10} \times 2^{10} \times 2^{10} = 2^{30} = 1\,073\,741\,824$, soit plus d'un milliard : les professionnels de la vidéo y gagnent des dégradés sans marches visibles. À l'inverse, une image indexée sur 4 bits ne garde que $2^4 = 16$ couleurs, comme les premières interfaces graphiques.

5 Poids d'une image numérique

Méthode : calculer le poids brut d'une image

Le poids *non compressé* d'une image matricielle vaut

$$\text{poids (octets)} = \frac{\text{largeur} \times \text{hauteur} \times \text{profondeur (bits)}}{8},$$

la profondeur étant de 1 bit (noir et blanc), 8 bits (niveaux de gris) ou 24 bits (RVB). Les formats compressés (JPEG pour les photos, PNG pour les graphiques) réduisent ce poids d'un facteur qui dépend du contenu, typiquement de 5 à 20 pour un JPEG de qualité courante.

Format	Usage	Compression	Facteur typique
JPG	photographies	avec perte	$\times 5$ à $\times 20$
PNG	captures, logos, transparence	sans perte	$\times 2$ à $\times 3$
GIF	petites animations	indexée (256 couleurs)	variable
SVG	dessins vectoriels	sans objet (équations)	non applicable

Table 6.4 – Les formats d'image et leur comportement à l'enregistrement

Exemple résolu 6.8 : le poids d'une photographie

La photo de 4000×3000 pixels en RVB (24 bits) pèse, non compressée :

$$\frac{4000 \times 3000 \times 24}{8} = \frac{288\,000\,000}{8} = 36\,000\,000 \text{ octets} = 36 \text{ Mo}.$$

Enregistrée en JPEG de qualité courante, elle pèse environ 3,6 Mo, soit dix fois moins : la compression sacrifie des détails imperceptibles à l'œil. C'est le poids que l'on télécharge réellement, et celui qui détermine le temps d'envoi : à 2 Mo par seconde, 3,6 Mo partent en moins de deux secondes.

Exemple résolu 6.9 : le poids d'une page scannée

Une page A4 (21 cm sur 29,7 cm) est scannée à 300 dpi en niveaux de gris (8 bits). Ses dimensions en pixels valent $\frac{21 \times 300}{2{,}54} \approx 2480$ et $\frac{29{,}7 \times 300}{2{,}54} \approx 3508$, soit $2480 \times 3508 \approx 8\,700\,000$ pixels. En niveaux de gris, le poids brut vaut 8 700 000 octets, soit 8,7 Mo ; en noir et blanc (1 bit par pixel), il tombe à $\frac{8\,700\,000}{8} \approx 1{,}1$ Mo : la raison pour laquelle un service de scan de documents choisit le noir et blanc pour le texte.

Le calcul du poids se lit aussi pixel par pixel : une profondeur de 24 bits, c'est 3 octets par pixel, de sorte que chaque ligne de 4000 pixels d'une photo plein format occupe $4000 \times 3 = 12\,000$ octets, et les 3000 lignes totalisent bien $12\,000 \times 3000 = 36$ millions d'octets. Ajouter un quatrième canal (l'opacité, notée alpha, dans le codage RGBA) porterait la profondeur à 32 bits et le poids brut à 48 Mo : la transparence se paie en octets.

Remarque

[du poids au temps d'envoi] Le poids d'une image se convertit en temps de transmission. Une photo de 36 Mo non compressée représente $36 \times 8 = 288$ mégabits ; sur une liaison à 10 Mbit/s, son envoi demande $288/10 \approx 29$ secondes. Compressée en JPEG à 3,6 Mo, soit 28,8 mégabits, la même photo part en moins de 3 secondes. C'est cette arithmétique (bits, octets, débit) qui explique pourquoi les messageries compressent systématiquement les images avant l'envoi.

Ce qu'il faut retenir

Un vecteur est une liste de nombres, une matrice un tableau ; le produit matrice-vecteur applique les transformations de base (échelle $\begin{pmatrix} k & 0 \\ 0 & k \end{pmatrix}$, rotation $\begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}$) aux pixels d'une image. Image matricielle (JPG, PNG : grille de pixels) et image vectorielle (SVG : équations) ne réagissent pas de même au zoom. La définition compte les pixels ($4000 \times 3000 = 12$ Mpx), la résolution les répartit (300 dpi $\Rightarrow$ 33,9 cm de large), le rapport 16 :9 ou 4 :3 fixe les proportions. En RVB 24 bits : $2^{24} = 16\,777\,216$ couleurs, chaque canal écrit en hexadécimal, $(164, 78, 20) =$ `#A44E14`. Le poids brut vaut largeur $\times$ hauteur $\times$ profondeur / 8 octets : 36 Mo pour la photo de 12 Mpx, ramenés à 3,6 Mo par le JPEG.